

바이브(Vibe) 코딩

• 바이브(Vibe) 코딩

바이브 코딩(Vibe Coding)은 사람이 직접 소스 코드를 타이핑하는 대신, 생성형 AI(LLM)와 자연어로 대화하며 원하는 결과물(앱이나 웹사이트)의 '느낌(Vibe)'과 기능을 설명해 개발하는 새로운 소프트웨어 개발 방식이다.

1. 바이브 코딩 개발 도구

바이브 코딩 개발 도구들은 모두 AI 기반 코딩 환경을 구축하는 핵심 기술 및 소프트웨어들이지만, 구동 방식(에디터형, 터미널형)과 출시 주체에 따라 뚜렷한 차이점을 가지고 있다. 현재 가장 많이 활용되는 대표적인 바이브 코딩 개발 도구는 다음과 같다.

- 1) Cursor(커서): 가장 대중적으로 쓰이는 AI 에디터이다. 기존 VS Code 환경과 완벽히 호환되며, 파일 전체의 맥락을 이해하고 여러 파일에 걸쳐 코드를 자동 수정 및 리팩토링하는 능력이 뛰어나다.
- 2) Claude Code(클로드 코드): OpenAI 출신의 핵심 연구원들이 설립한 앤트로픽(Anthropic)에서 개발하였으며 개발자와 대화하며 코드 베이스 전체를 분석하고 개선안을 제시하는 등 고도화된 AI 코딩 지원에 강하다.
- 3) 코덱스(Codex): 챗GPT(ChatGPT)로 잘 알려진 오픈AI(OpenAI)에서 개발하였으며 프로젝트 폴더 전체를 읽고, 스스로 테스트를 실행하고, 디버깅 후 깃허브(GitHub) 브랜치 생성 및 풀 리퀘스트(PR) 오픈까지 전 과정을 자율적으로 처리한다.
- 4) 안티그래비티(Antigravity): 구글에서 만들었으며 단순히 코드를 추천해 주는 비서 역할을 넘어, 사용자가 말 한마디만 하면 스스로 계획을 세우고 컴퓨터의 파일과 터미널을 직접 조작해 앱을 완성하는 '에이전트 퍼스트(Agent-first)' 환경을 제공한다.

2. 필요한 프로그램 설치

커서(Cursor)를 활용해 바이브(Vibe) 코딩을 할 때 반드시 설치해야할 프로그램들을 다음과 같다.

- 1) VS-Code기반으로 만들어진 에디터를 설치한다. 단순한 챗봇연동을 넘어 개발 환경 전체가 AI 워크플로우에 최적화된 기능들을 제공한다.

```
https://cursor.com/ko/download
```

- 2) AI가 생성한 자바스크립트 코드를 컴퓨터에서 실행하고, 필요한 개발도구와 패키지를 다운로드하여 구동하기 위해서 Node.js를 설치한다.

```
https://nodejs.org/ko/download
```

3. 커서(Cursor) 개발 환경 세팅

Cursor 통합개발환경(IDE)을 효율적으로 사용하기 위한 개발 환경 세팅 방법을 다음과 같다.

- 1) 아래 메뉴에서 통합개발환경(IDE)의 색상 테마를 밝은 모드, 어두운 모드로 변경할 수 있다.

```
[File]-[Preferences]-[Themes]-[Color Theme]
```

- 2) 아래 메뉴에서 'gpt-4o-mini'만 활성화 시키고 나머지 모델들은 모두 비활성화 한다. Composer창도 Agent를 Auto 모드로 변경한다.

```
[오른쪽 상단 톱니바퀴]-[왼쪽 Models]
```

- 3) 아래 메뉴에서 현재 남아있는 Credit 상황을 간단히 확인할 수 있다.

```
[오른쪽 상단 톱니바퀴]-[왼쪽 Plan & Usage]-[Total]
```

4. Auto RUN Mode

[오른쪽 상단 톱니바퀴]-[Agent]-[Auto Run]-[Auto Run Mode]에 아래와 같이 세 가지 모드가 있다.

구분	Ask Everytime	Use Allowlist(추천)	Run Everything
개념	모든 명령을 실행하기 전 사용자 승인필요	허용 목록에 있는 명령만 자동 실행	승인 없이 모든 명령을 완전 자동실행
안전성	최상(모든 명령을 검증 가능)	상(필요한 명령만 선별적 허용)	하(위험 명령어 노출 위험)
편의성	하(매번 클릭해야 하므로 흐름이 끊김)	상(안전하면서도 반복 작업 자동화)	최상(완전히 자율 디버깅 및 빌드)
권장대상	보안이 중요한 기업 프로젝트	대부분의 일반적인 웹/앱 개발 환경	독립된 샌드박스나 테스트 환경

• 자기소개 홈페이지

홈페이지는 웹 브라우저를 켜고 때 자동을 열리는 웹페이지나 특정 웹 사이트(Website)의 주소를 입력했을 때 가장 먼저 보여주는 메인 페이지(Main Page)를 뜻하지만 국내에서는 여러 개의 웹페이지와 콘텐츠가 모여 있는 웹 사이트 전체를 홈페이지라고 부르는 경우 많다.

[c:]-[data]-[cursor] 폴더에서 **my-homePage** 폴더를 생성하여 자기소개 페이지 프로그램을 작성한다.

1. HTML/CSS/자바스크립트

브라우저에서 홈페이지 HTML 파일 결과를 보기위해 [Search Extensions]에서 **open in browser** 프로그램을 설치한다.

목표: 아래 프로필 내용을 참고하여 풍부하게 가상의 더미 데이터를 생성한 후 마크다운 파일 '**profile.md**'로 저장한다.

[프로필]

이름: 홍길동

주소: 서울특별시 강남구 대치동 희망 아파트 109동 2104호

전화번호: 더미 데이터 생성

회사: 한국 소프트웨어 인재개발원

경력: 10년간 IT 기획/개발자로 일했음

목표: **@profile.md** 파일 내용을 참고하여 순수하게 HTML로만 구성된 자기소개 페이지 하나를 작성한다.

[유의사항]

1. 웹 페이지를 만들 때 한글이 깨지지 않도록 한다.
2. 디자인은 나중에 적용할 예정이므로 CSS는 적용하지 않는다.

목표: 생성된 HTML 파일을 요구사항을 참고하여 CSS를 추가해서 예쁘게 수정한다.

[요구사항]

1. 깔끔하고 세련되게 디자인하고 전체적인 색상을 파랑색으로 한다.
2. 제목은 네이비 배경에 흰색 글자로 출력한다.
3. 한글의 폰트는 구글(<https://fonts.google.com/>)의 'Noto Sans Korean'로 설정한다.

목표: 생성된 HTML 파일을 자바스크립트를 이용해 인터랙티브(Interactive)한 기능을 추가한다.

[요구사항]

1. 자기소개 내용을 오른쪽으로 넘기는 슬라이드 형태로 만든다.
2. Light 모드와 Dark 모드로 변환되는 기능을 추가한다.

프로필 내용을 자연스럽게 위에서 아래로 가는 방식으로 변경한다.

2. 디자인 적용하기

- Tailwind(테일윈드): HTML 코드 안에서 클래스 이름을 조합하는 것만으로 화면을 디자인할 수 있는 CSS 프레임워크이다.
- shadcn/ui(샬드씨엔/유아이): 버튼, 카드, 팝업창을 처음부터 만들 필요 없게 미리 다 만들어 둔 만능 부품 모음집이다.
- tweakcn(트웁씨엔): 웹 사이트의 색상과 테두리 모양을 마우스 클릭만으로 바꾸고 코드를 뽑아내는 온라인 디자인 작업실이다.
[<https://tweakcn.com/>]-[Brows Community]-[All Themes]-[Portfolio theme]-[View]-[Code]-[Copy] 내용을 'global.css'에 저장한다.

목표: 라이브러리를 사용하여 HTML 파일 디자인을 변경한다.

1. Tailwind CSS CDN(여백 및 레이아웃)

- 요소들 사이에 답답하지 않도록 풍부한 여백을 적용한다.
- 카드에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.

2. tweakcn 테마 설정(전체 색상 및 테두리)

- 전역 스타일로 **@global.css** 파일을 반영하여 사이트 전체를 통일감이 있도록 한다.

3. 컴포넌트 교체(부품 업그레이드)

- Tailwind CSS를 조합해서 '**shadcn/ui**' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
- 아이콘은 '**lucide**' CDN 라이브러리를 사용해 자기소개에 어울리는 그래픽 요소들로 배치한다.

• 과일 가게 홈페이지

서버와 데이터베이스 사용하지 않고 로컬 환경에서 작동하는 홈페이지를 HTML, CSS, JavaScript, 각종 디자인 라이브러리로 작성한다.

[c:]-[data]-[cursor] 폴더에서 **fruit-homepage** 폴더를 생성하여 과일 가게 홈페이지를 작성한다.

1. 전체적인 레이아웃 설계(뼈대 만들기)

목표: 서버와 데이터베이스가 없이 로컬 환경에서만 작동하는 '온라인 과일가게' 메인 홈페이지를 디자인을 최소화해서 작성한다.

[기술스택]: HTML5 + CSS + JavaScript

[페이지 레이아웃]

1. 네비게이션 바(Navigation Bar): 좌측에 텍스트 로고, 중앙에 메뉴(홈, 제철과일), 우측에 '장바구니 배지(0)' 표시
2. 히어로 섹션(Hero Section): 큰 제목("신선한 과일 마켓")과 간단한 설명, 그리고 [구경하기] 버튼이 있는 상단 영역
3. 메인 콘텐츠(Main Contents): 2행 4열로 구성된 그리드 레이아웃으로 8개 과일 아이템 들어갈 상품 카드 구역
 - 상품카드 내부: 과일이름, 가격(원화), [장바구니 담기] 버튼
4. 사이드 바(Side Bar): 화면 우측에 현재 장바구니에 담긴 과일 목록과 총 금액이 표시되는 구역
5. 푸터(Footer): 터미 데이터로 생성

[기능 요구사항]

- '장바구니 담기' 버튼을 누르면 상단 배지의 숫자와 장바구니 영역의 총 금액이 실시간으로 증가하는 자바스크립트로 작성한다.

[디자인 요구사항]

- 테두리는 투박한 검은색 선, 배경은 흰색, 글씨는 검은색 위주로만 아주 기본적으로만 처리한다. 세련된 디자인은 2단계에서 처리한다.

2. 디자인 도구 3종 세트 적용 (환골탈태하기)

• Tweakcn 사이트에서 배경색과 Radius를 조절하여 생성된 코드를 복사하여 **global.css**에 붙여 넣는다.

- 1) [https://tweakcn.com/]-[Start Customizing]-[Colors]-[PRIMARY]-[Background] 색상을 변경한다.
- 2) [https://tweakcn.com/]-[Start Customizing]-[Other]-[RADIUS] 모서리 둥근 정도를 조절한다.

목표: 1단계에서 만든 레이아웃 뼈대와 장바구니 기능은 그대로 유지하면서 아래 도구를 활용해 트렌디한 디자인으로 변경한다.

1. tweakcn 테마 설정(전체 색상 및 테두리)

- @global.css 스타일을 반영하여, 사이트 전체가 **싱그러운 과일가게 홈페이지** 톤으로 변경한다.

2. 컴포넌트 교체(부품 업그레이드)

- Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
- 아이콘은 'lucide' CDN 라이브러리를 사용해 **과일쇼핑몰**에 어울리는 그래픽 요소들로 배치한다.

3. Tailwind CSS CDN 세부 스타일링(여백 및 디테일)

- 요소들 사이에 답답하지 않도록 풍부한 여백을 적용한다.
- 상품 카드에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
- 과일 이미지가 들어갈 카드상자에는 이미지를 넣기 전에 과일아이콘으로 출력한다.

3. 홈페이지 다듬기

- 1) 히어로 섹션에 배경으로 사용할 이미지를 생성한다.

우리 홈페이지와 어울리는 최적화된 가벼운 히어로 이미지(images/hero.webp)를 생성한다.

- 2) [Visual Studio Code]-[View]-[Command Palette]-[Open Browser]에서 HTML을 출력하여 히어로 섹션을 클릭한다.

- 3) 생성한 이미지를 히어로 섹션 배경으로 바꿔주는 프롬프트를 실행한다.

<section> 영역에 @images/hero.jpg 이미지를 멋지게 배경으로 출력한다.

- 4) 8개의 과일 이미지를 생성하여 상품 카드 이미지 영역에 출력하는 프롬프트를 실행한다.

최적화된 가벼운 과일 이미지 webp 파일 8개를 생성하여 상품 카드 이미지 영역에 출력한다.

• 인테리어 업체 홈페이지

다른 컴퓨터에서도 우리 인테리어 업체 홈페이지에 접속하기 위하여 Express 프레임워크로 웹 서버를 생성한 후 웹 사이트를 구축한다.

[c:]-[data]-[cursor] 폴더에서 **interior-hompage** 폴더를 생성하여 인테리어 업체 홈페이지를 작성한다.

1. 전체적인 레이아웃 설계

목표: Express로 웹 서버를 생성한 후 디자인을 최소화해서 인테리어업체 '디자인 홈'의 메인 홈페이지를 작성한다.

[기술스택]

1. 백엔드 서버(BackEnd Server): Express
2. 프론트엔드 템플릿(FrontEnd Template): EJS
3. 서버 구동: `npm run dev`

[페이지 레이아웃]

1. 네비게이션 바(Navigation Bar): 좌측에 텍스트 로고, 우측에 '시공 사례', '진행 방식', '견적문의' 표시
2. 히어로 섹션(Hero Section): 큰 제목("깔끔하고, 부담 없는 비용으로...")과 간단한 설명, 그리고 [구경하기] 버튼이 있는 상단 영역
3. 메인 콘텐츠(Main Contents): 1행 4열로 구성된 그리드 레이아웃으로 각 그리드 아이템은 업체 결과물이 들어갈 카드상자 구역
 - 카드상자 내부: 결과물 이미지, 결과물 이름, 결과물 간단한 설명
4. 푸터(Footer): 저작권, 회사 주소, 전화번호를 간단하게 터미 데이터로 생성

[디자인 요구사항]

- 테두리는 투박한 검은색 선, 배경은 흰색, 글씨는 검은색 위주로만 아주 기본적으로만 처리한다. 세련된 디자인은 2단계에서 처리한다.

2. 디자인 적용하기

[https://getdesign.md/]-[Starbucks]-[DESIGN.md]-[Copy] 내용을 복사하여 DESIGN.md 파일에 붙여 넣는다.

목표: 1단계에서 만든 레이아웃을 그대로 유지하면서 아래 도구를 활용에 트렌디한 디자인으로 변경한다.

[디자인 기술스택]

- Tailwind CSS CDN, lucide CDN

[디자인 요구사항]

1. @DESIGN.md 디자인 기획서를 반영하여 사이트 전체가 **모던하고 세련된 인테리어 홈페이지** 톤으로 변경한다.
2. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
3. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
4. 아이콘은 'lucide' 라이브러리를 사용해 **인테리어업체** 홈페이지에 어울리는 그래픽 요소들로 배치한다.

3. 프로젝트 배포

Vercel(버셀)은 웹 애플리케이션을 빠르고 쉽게 배포하고 호스팅할 수 있게 해주는 클라우드 플랫폼이다. 복잡한 서버 설정 없이 깃허브 같은 저장소를 연동하기만 하면 자동으로 웹을 빌드하고 전 세계에 배포해 준다. Vercel 사이트(<https://vercel.com/login>)에서 Hobby(무료 개인계정) 플랜을 선택해 Google 또는 Github와 연동 로그인하면 간단하게 가입이 끝나고 사용이 가능해진다.

- vercel --prod: 프리뷰(Preview) 배포가 아닌, 실제 서비스 도메인으로 변경사항을 즉시 반영하여 배포한다.

```
C:/현재 프로젝트>npm install vercel -g
```

```
C:/현재 프로젝트>vercel login
```

```
C:/현재 프로젝트>vercel
```

```
Set up and deploy "C:\data\cursor project\memo" ? (Y/n) → 그냥엔터
```

```
Which scope should contain your project? (Use arrow keys) → 내 계정 이름 확인 후 엔터
```

```
> zoe cho's projects
```

```
Which scope should contain your project? zoe cho's projects
```

```
Link to existing project? (y/N) → (기존 프로젝트 연결 여부) 그냥엔터
```

```
What's your project's name? → 프로젝트 이름(반드시 소문자) 설정 후 엔터
```

```
In which directory is your code located? ./ → 현재 위치 파일이 맞으므로 엔터
```

```
> Auto-detected Project Settings for Express
```

```
Want to modify these settings? (y/N) → (빌드 옵션 수정 여부) 프레임워크를 알아서 감지하므로 그냥엔터
```

• 게임 홈페이지

다른 컴퓨터에서도 게임을 플레이할 수 있도록 Express 프레임워크를 사용하여 웹 서버를 생성한 후 게임 웹 사이트를 구축한다.

[c:]-[data]-[cursor] 폴더에서 `game_homepage` 폴더를 생성하여 두 게임을 플레이할 수 있도록 게임 홈페이지를 작성한다.

1. 홈페이지 레이아웃 설계 및 디자인 적용

[https://getdesign.md/]-[Apple]-[DESIGN.md]-[Copy] 내용을 복사하여 DESIGN.md 파일에 붙여 넣는다.

목표: Express로 웹 서버를 생성한 후 게임을 위한 'ENJOY' 메인 홈페이지 레이아웃을 설계하고 디자인을 적용한다.

[기술스택]

1. 서버: Express, EJS
2. 디자인: Tailwind CSS CDN, lucide CDN
3. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 네비게이션 바(Navigation Bar): 좌측에 텍스트 'ENJOY' 로고, 우측에 '문의사항', '게시판', '로그인' 표시 (우측 메뉴는 나중에 연결)
2. 히어로 섹션(Hero Section): 큰 제목('애플, 알카노이드 게임')과 간단한 설명, 그리고 '구경하기' 버튼이 있는 상단 영역
3. 메인 콘텐츠(Main Contents): 1행 2열로 구성된 그리드 레이아웃으로 각 그리드 아이템은 카드상자로 구성
 - 카드상자 내부: '애플 게임', '알카노이드 게임' 게임 이름과 게임 간단한 설명
4. 푸터(Footer): 저작권, 회사 주소, 전화번호를 간단하게 터미 데이터로 생성

[디자인 요구사항]

1. @DESIGN.md 디자인 기획서를 반영하여 사이트 전체가 **다이나믹(Dynamic)한 게임 홈페이지** 톤으로 변경한다.
2. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
3. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
4. 아이콘은 'lucide' 라이브러리를 사용해 **게임** 홈페이지에 어울리는 그래픽 요소들로 배치한다.

2. 사과(Apple) 게임

사과 게임 규칙을 검색하여 `apple\_game.md` 마크다운 파일에 저장한다.

[애플게임 요구사항]

- @apple_game.md 게임 규칙 기획서를 참고해서 프로그램 페이지를 작성한 후 '애플 게임' 상자를 클릭하면 해당 페이지로 이동한다.

[애플게임 추가 요구사항]

1. 페이지 상단 왼쪽에 점수, 남은 시간, 선택 압, 상단 오른쪽에 게임시작 버튼, 게임종료 버튼을 상자 안에 예쁘게 한 줄로 배치하여 추가한다.
2. 사과 아이콘을 상자에 80%로 채우고 글자를 사과 아이콘 안에 출력한다.
3. 게임종료 버튼을 클릭한 경우 '게임을 종료하시겠습니까?' 모달창(헤더: 기본 버튼과 같은 색상 테마)을 띄워 질의 후 '예'인 경우 홈페이지로 이동한다.

3. 알카노이드(Arkanoid) 게임

알카노이드 게임 규칙을 검색하여 `arkanoid\_game.md` 마크다운 파일에 저장한다.

[알카노이드게임 요구사항]

- @arkanoid_game.md 게임 규칙 기획서를 참고해서 프로그램 페이지를 작성한 후 '알카노이드 게임' 상자를 클릭하면 해당 페이지로 이동한다.

[알카노이드 추가 요구사항]

단조로운 벽돌 패턴을 깨고 내부도와 속성을 가진 특수 벽돌들을 추가한다.

1. 일반 벽돌: 노란색이나 초록색으로 1대 맞으면 깨짐.
2. 단단한 벽돌: 회색 벽돌로, 3번 맞아야 깨짐. 맞을 때마다 금이 가는 시각적 효과(Cracked effect)를 준다.
3. 폭발 벽돌: 빨간색 벽돌로, 파괴될 때 상하좌우 인접한 벽돌을 함께 파괴하는 범위 데미지를 준다.
4. 보이지 않는 벽돌: 공이 처음 부딪히기 전까지는 투명했다가, 맞으면 나타나면서 2번째 타격에 깨지게 한다.

• AGENTS.md 와 Agent Skill

AGENTS.md와 Agent Skill은 AI 코딩 에이전트(예: Claude Code, Cursor 등)가 프로젝트를 더 잘 이해하고 특정 작업을 전문적으로 수행할 수 있도록 돕는 설정 및 지식 확장 체계이다. 두 개념은 AI 에이전트에게 맥락을 전달하는 방식과 목적에서 명확한 차이가 있다.

구분	AGENTS.md (에이전트 정의 문서)	Agent Skill (에이전트 스킬)
정의	프로젝트 단위의 전반적인 작업 규칙 문서	에이전트가 특정 작업을 수행할 수 있도록 부여된 기능적 도구
범위	특정 프로젝트에 종속적	여러 프로젝트에서 독립적으로 재사용 가능
형태	텍스트 파일 (.md)	skill.md 파일이 포함된 하나의 폴더(디렉터리) 구조
방식	프롬프트 입력 시 에이전트가 항상 기억하고 참조하는 운영규칙	에이전트가 작업 중 필요할 때마다 스스로 호출하여 사용

1. 프로젝트 개발 시나리오

우리 회사는 홈페이지를 제작해주는 회사이다. 홈페이지에 사용되는 기술스택과 디자인 라이브러리는 직원 모두가 통일하여 사용하고 작성할 홈페이지의 카테고리(예: 의료기관, 교육기관, 쇼핑몰)에 따라 다른 메뉴와 다른 디자인으로 페이지를 생성하는 프로젝트를 작성한다.

[c:]-[data]-[cursor] 폴더에서 `agents-skills` 폴더를 생성하여 AGENTS.md와 Agent Skill을 이용한 홈페이지를 작성한다.

2. 프로젝트 작업 규칙 문서(AGENTS.md) 작성

목표: `AGENTS.md` 파일을 생성해서 이 문서에 프로젝트에서 사용할 기술 스택과 규칙을 정의하는 지침서를 작성한다.

[공통 기술스택]

1. HTML, CSS, JavaScript

2. 'Tailwind' CDN, 'lucide' CDN, Tailwind CSS를 사용한 'shadcn/ui'와 같은 디자인

[공통 개발규칙]

1. 로컬에서 실행될 단일 페이지로 구성

2. 모바일과 데스크톱 화면을 모두 지원 가능한 반응형 필수

3. 병원 홈페이지 스킬(Agent Skill) 작성

/create-skill

1. Skill name: `clinic-skill-builder`

2. Description

- 사용자가 병원, 의원, 치과, 피부과, 한의원 등 의료 기관의 홈페이지 제작을 요청했을 때 작동하는 프로젝트 스킬이다.

- 의료 기관에 필요한 신뢰감 있고 깔끔한 UI 구조와 업종별 필수 메뉴를 자동으로 주입한다.

3. Trigger Keywords

- 병원, 의원, 메디컬, 치과, 피부과, 안과, 성형외과, 내과, 한의원, 진료, 예약

4. Skill Prompt (AI 행동 지침)

[디자인 요구사항]

1. 주 색상: 청결함과 신뢰감을 주는 스카이블루 또는 에메랄드그린을 기본으로 사용한다.

2. 배경 색상: 무조건 깨끗한 화이트 또는 아주 밝은 그레이를 채택한다.

[네비게이션 바]

- 좌측에는 로고와 홈페이지 제목을 중앙에는 [병원소개]-[의료진 소개]-[진료안내]-[오시는 길 / 예약], 우측에는 전화번호

[메인 콘텐츠]

1. 히어로 섹션: '환자를 최우선으로 생각합니다'와 같은 신뢰감을 주는 슬로건과 '빠른 예약하기' 버튼을 배치한다.

2. 병원소개(병원소개 글, 병원사진)과 의료진 소개(사진, 의료진 이름, 졸업대학 전공분야)를 깔끔하게 배치한다.

3. 진료시간 안내: 평일, 토요일, 점심시간, 일요일/공휴일 휴무 여부를 명확하게 보여주는 표(Table) 형태의 UI를 제공한다.

4. 오시는 길: 주소, 전화번호, 주차 안내 문구와 함께 지도(Map) 영역을 더미(Dummy) 박스로 시각화한다.

5. 하단 푸터(Footer): 대표전화번호, 이메일 주소, 저작권(Copyright)을 더미 데이터로 출력

6. 아이콘은 'lucide' 라이브러리를 사용해 병원 홈페이지에 어울리는 그래픽 요소들로 배치한다.

7. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.

'튼튼한 정형외과' 병원 홈페이지를 `clinic.html` 파일로 만들어줘.

4. 교육기관 홈페이지 스킬(Agent Skill) 작성

/create-skill

1. Skill name: **education-skill-builder**

2. Description

- 사용자가 학교, 학원, 교육 기관, 인터넷 강의, 클래스 등의 홈페이지 제작을 요청했을 때 작동하는 **프로젝트** 스킬이다.
- 교육 기관에 걸맞은 체계적인 커리큘럼 구조와 학습 욕구를 자극하는 UI를 자동으로 주입한다.

3. Trigger Keywords

- 학교, 대학교, 고등학교, 학원, 코딩학원, 영어학원, 교육, 커리큘럼, 강좌, 클래스, 수강

4. Skill Prompt (AI 행동 지침)

[디자인 요구사항]

1. 주 색상(Primary Color): 지적이고 신뢰감을 주는 닙네이비와 포인트를 줄 수 있는 웜옐로우를 조합한다.
2. 레이아웃: 정보가 많아 보이므로 그리드(Grid)와 카드(Card) 형태를 적극 활용하여 정돈된 느낌을 준다.

[네비게이션 바]

- 좌측에는 로고와 홈페이지 제목, 중앙에는 [학원 소개]-[강사 프로필]-[교육 과정(커리큘럼)]-[수강 후기 / 공지사항], 우측에는 '수강상담' 버튼

[메인 콘텐츠]

1. 히어로 섹션: '당신의 성장을 설계합니다.'와 같은 전문성을 주는 슬로건과 '수강 상담하기' 버튼 배치한다.
2. 강사진 소개 카드: 강사 사진(데미), 이름, 대표 약력을 깔끔하게 정렬한 3열 카드 레이아웃
3. 커리큘럼 타임라인: 수강생이 어떤 순서로 공부하는지 한눈에 볼 수 있는 타임라인(Timeline) 또는 스텝(Step)바 UI 제공
4. 수강 후기 회전(Rolling) 보드: 실제 수강생들의 생생한 한 줄 평가 별점이 들어간 리뷰섹션 배치.
5. 하단 푸터(Footer): 대표전화번호, 이메일 주소, 저작권(Copyright)을 데미 데이터로 출력
6. 아이콘은 'lucide' 라이브러리를 사용해 교육기관 홈페이지에 어울리는 그래픽 요소들로 배치한다.
7. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.

'스마트' 영어학원 홈페이지를 `education.html` 파일로 만들어줘.

5. 쇼핑몰 홈페이지 스킬(Agent Skill) 작성

/create-skill

1. Skill name: **shopping-skill-builder**

2. Description

- 사용자가 쇼핑몰, 마켓, 커머스, 스토어 등 상품 판매 목적의 웹사이트 제작을 요청했을 때 작동하는 **프로젝트** 스킬이다.
- 상품이 돋보이는 그리드 레이아웃, 장바구니 UI, 직관적인 구매 전환(CTA) 버튼 구조를 자동으로 주입한다.

3. Trigger Keywords

- 쇼핑몰, 마켓, 스토어, 커머스, 상점, 쇼핑, 옷가게, 굿즈샵, 편집숍, 상품리스트, 장바구니

4. Skill Prompt (AI 행동 지침)

[디자인 요구사항]

1. 주 색상: 구매 버튼이나 할인율 등 시선을 끌어야 하는 포인트에는 활력 있는 오렌지 또는 세련된 블랙을 사용한다.
2. 배경 색상: 상품 이미지가 가장 돋보이도록 잡음이 없는 순수한 화이트 배경을 사용한다.

[네비게이션 바]

- 좌측에는 로고 , 우측에는 [SHOP(상품)]-[ABOUT]-[EVENT]-[장바구니]-[마이페이지] 아이콘 배열 구조를 상단 헤더에 반드시 포함한다.

[메인 콘텐츠]

1. 히어로 섹션: 비주얼 배너(Carousel:캐러셀): 화면 전체를 채우는 대형 기획전 배너 이미지(데미)와 '시즌 오프 최대 50% 세일' 같은 카피라이팅 배치
2. 상품 격자 리스트 (Product Grid): 모바일에서는 2열, 데스크톱에서는 4열로 정렬된 상품 카드 섹션 구성
 - 상품 카드 내부 요소: 상품 이미지(데미), 브랜드명(작게), 상품명, 할인율(레드 컬러) 및 판매가, '장바구니 담기' 킷 버튼을 반드시 포함
3. 하단 푸터(Footer): 쇼핑몰 필수 정보인 사업자 등록번호, 통신판매업 신고번호, 고객센터 운영시간(1:1 톡 문의 버튼)을 가상 데이터로 하단에 배치
4. 아이콘은 'lucide' 라이브러리를 사용해 **인테리어업체** 홈페이지에 어울리는 그래픽 요소들로 배치한다.
5. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가

'오버핏 스튜디오' 패션 쇼핑몰 홈페이지를 `shopping.html` 파일로 만들어줘.

• 데이터베이스

데이터베이스(Database)는 수많은 정보를 컴퓨터 시스템에 안전하게 저장하고, 필요할 때 언제든지 쉽고 빠르게 찾아 쓸 수 있도록 정리해 놓은 '디지털 정보 창고'이다. 프로젝트 개발에 사용할 데이터베이스 기술 스택은 아래와 같다.

1. SQLite(에스큐엘라이트)

SQLite는 복잡한 설치 과정 없이 내 컴퓨터 프로젝트 폴더 안에 단 하나의 파일(.db) 형태로 모든 데이터를 저장하는 가장 가볍고 단순한 데이터베이스이다. 보통 데이터베이스를 쓰려면 별도의 대형 서버 프로그램을 설치하고 컴퓨터 설정을 해야 하지만, SQLite은 그 모든 과정을 생략할 수 있어 초보자 실습 및 가벼운 프로그램 개발에 전 세계 1위로 쓰인다.

2. Prisma(프리즈마) Object-Relational Mapping

복잡하고 어려운 데이터베이스 명령어(SQL)를 몰라도 우리가 평소 쓰던 자바 스크립트나 파이썬 코드로 데이터베이스를 다룰 수 있게 해주는 라이브러리 도구이다. Prisma ORM은 중간에 서서 자바스크립트 객체를 데이터베이스 테이블로 자동 변환해주고 반대로 창고에서 데이터를 꺼낼 때는 테이블에 저장된 데이터를 자바스크립트 객체로 변환해서 개발자에게 전달해 준다.

[c:]-[data]-[cursor] 폴더에서 **database-lab** 폴더를 생성하여 데이터베이스 관련 프로그램들을 작성한다.

1. 메인(홈) 페이지 작성

[https://getdesign.md/]-[Starbucks]-[DESIGN.md]-[Copy] 내용을 복사하여 DESIGN.md 파일에 붙여 넣는다.

목표: 아래 기술스택과 내용을 참고하여 'DATABASE LAB' 위한 메인 페이지를 작성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 데이터베이스: SQLite + Prisma ORM (데이터베이스 이름: dev.db)
3. 디자인: 'Tailwind' CDN, 'Lucide' CDN
4. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 필수
2. 네비게이션 바(Navigation Bar): 좌측에 텍스트 로고, 우측에 '공지사항', '고객센터', '로그인' 버튼 (각 페이지는 나중에 제작예정)
3. 메인 콘텐츠(Main Contents): 1행 4열의 그리드 레이아웃으로 각 그리드 아이템은 카드상자로 구성
4. 카드상자 내부: '메모앱', '마크다운 프리뷰', '체크리스트', '프롬프트 관리' 제목과 간단한 설명
5. 푸터(Footer): 저작권, 회사 주소, 전화번호를 간단하게 더미 데이터로 생성

[디자인 요구사항]

1. @DESIGN.md (Starbucks) 디자인 마크다운 파일을 참고하여 테마를 설정한다.
2. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
3. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
4. 아이콘은 'Lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.

2. 메모앱 페이지 작성

목표: 메모앱 관리 페이지를 작성하고 메인 콘텐츠 '메모앱' 카드 메뉴와 연결한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 'Memo' 테이블 스키마를 정의한다.
2. 테이블 필드는 id, content(내용), tags(태그), createdAt(작성일)로 구성한다.

[기능 및 UI/UX 요구사항]

1. 메모 작성 창
 - 사용자가 메모를 입력하고 '저장' 버튼을 누르면 DB에 즉시 저장하고 전체 목록을 화면에 출력한다.
 - 메모 내용 입력창 아래에 '태그 입력창'을 따로 만든다. (예:"공부, 일상, 개발"처럼 쉼표로 나누어 여러 개 입력)
2. 태그 필터링 바
 - 화면 상단에 현재 등록된 모든 태그들을 중복 없이 추출해서 '태그' 버튼으로 출력한다.
 - 전체보기 버튼과 각 태그 버튼을 클릭하면, 해당 태그가 포함된 메모만 필터링하여 출력하는 기능을 구현한다.
3. 메모 목록
 - 2열의 그리드 레이아웃으로 구성하고 각 메모 카드 우측 상단에는 메모를 지울 수 있는 '삭제(X)' 버튼을 배치한다.
 - 삭제 시 '정말로 삭제하시겠습니까?' 모달창을 띄워 질의하고 '예'인 경우 삭제한다.
4. 메모를 삭제하거나 저장할 때 툴팁 뒤는 듯 부드러운 애니메이션 효과를 적용하고 모든 모달창의 헤더를 기본 버튼의 배경색/글자색과 같게 설정한다.

3. 마크다운 프리뷰 페이지 작성

목표: 마크다운 프리뷰 관리 페이지를 작성하고 메인 콘텐츠 '마크다운 프리뷰' 카드 메뉴와 연결한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 'Documents' 테이블 스키마를 정의한다.
2. 테이블 필드는 id, content(내용), updatedAt(수정일)로 구성한다.

[기능 및 UI/UX 요구사항]

1. 중앙 좌측 사이드바 (20%)
 - '새 문서 만들기' 버튼: 새로운 문서를 생성한다.
 - 저장된 목록 리스트
 - 1) 특수문자를 제거한 내용의 일부 한 줄과, 수정일 출력, 클릭 시 본문 불러오기, 각 리스트 옆에 삭제[X] 버튼을 포함한다.
 - 2) 삭제 시 모달창에 내용을 '정말로 삭제하시겠습니까?' 질의하고 '예'인 경우 삭제한다.
2. 중앙 우측 입력창 및 프리뷰(80%): 두 개의 Tab으로 구성
 - 첫 번째 탭: 줄 번호가 표시되는 마크다운 텍스트 편집기
 - 두 번째 탭: 실시간 마크다운 렌더링 화면
3. 상단 톨바: '저장하기' 버튼, PDF 내보내기' 버튼 오른쪽에 배치
 - '저장하기' 버튼을 누르면 입력창 내용이 저장되고, 사이드바 목록을 즉시 갱신한다.
 - 'PDF 내보내기'
 - 1) 사용자가 'PDF 내보내기' 버튼을 누르면 프리뷰(Preview) 영역의 HTML 렌더링 결과물만 PDF 파일로 다운로드한다.
 - 2) PDF 변환 설정(html2pdf 옵션): A4 용지 규격, 여백(Margin) 사방 15mm~20mm 부여

4. 체크리스트 페이지 작성

목표: 체크리스트 관리 페이지를 작성하고 메인 콘텐츠 '체크리스트' 카드 메뉴와 연결한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 'Todos' 테이블 스키마를 정의한다.
2. 테이블 필드는 id, content(내용), completed(완료 유무), createdAt(작성일)로 구성한다.

[기능 및 UI/UX 요구사항]

1. 화면 상단에 오늘 날짜, 현재 진행률(완료된 일/ 전체 할 일%)을 시각적으로 보여주는 프로그레스 바를 배치한다.
2. 새로운 할 일을 입력할 수 있는 텍스트 input 창과 스타일리시한 '추가' 버튼을 배치한다.
 - 추가 버튼 클릭 시 데이터베이스에 저장하고 한다.
3. 하단: 할 일 목록 리스트(작성일 역순)를 예쁜 표로 출력
 - 완료된 항목: 체크박스가 체크되어 있고, 텍스트에 취소선(line-through)과 연한 회색 처리한다.
 - 미완료 항목: 일반 텍스트로 명확하게 보이게 해준다.
 - 각 항목 우측에는 휴지통 아이콘이나 '삭제' 버튼을 배치한다.
 - 삭제 버튼을 클릭한 경우 '정말로 삭제하시겠습니까?' 모달창을 띄워 질의하고 '예'인 경우 삭제한다.
4. 페이지네이션은 데스크톱인 경우 (itemsPerPage:5, visiblePages:10), 모바일인 경우 간단한 UI(이전, 다음)로 다르게 사용한다.

5. 프롬프트관리 페이지 작성

- 1) [구글]-[스프레드시트] 새로운 시트를 만들어 '프롬프트' 칼럼에 더미 데이터를 100개 정도 저장한다.
- 2) [공유]-[일반 액세스] '제한됨'에서 '링크가 있는 모든 사용자'로 설정을 변경한다.
- 3) [공유]-[링크 복사] 버튼을 클릭하여 링크주소를 복사해 사용한다.

목표: 프롬프트 관리 페이지를 작성하고 메인 콘텐츠 '프롬프트 관리' 카드 메뉴와 연결한다.

[데이터베이스 요구사항]

스프레드시트 주소(링크 공유함):

https://docs.google.com/spreadsheets/d/1Keg-g57IkV-kUnssMI2s_xi77dubhBzPZjrmuQuJFK0/edit?usp=sharing

[기능 및 UI/UX 요구사항]

1. 화면을 1열의 그리드 레이아웃으로 구성한다.
2. 그리드 아이템: 스프레드시트의 '행번호', '프롬프트'(3줄 이하), '복사하기' 버튼을 오른쪽에 배치한다.
3. 그리드 아이템 클릭 시 중간크기 모달창을 띄워 '프롬프트' 전체내용을 내용을 보여준다.
4. 페이지네이션은 데스크톱인 경우 (itemsPerPage:3, visiblePages:10), 모바일인 경우 간단한 UI(이전, 다음)로 다르게 사용한다.

6. 로그인/회원가입 페이지 작성

JWT(JSON Web Token)는 웹사이트나 앱에서 사용하는 '디지털 신분증' 또는 '위조 방지 입장권'이다. 로그인에 성공한 유저에게 서버가 발급해 주며, 유저는 이후 서비스를 이용할 때마다 이 토큰을 서버에 보여주어 자신이 로그인된 상태임을 증명한다.

개인 스킬인 경우 [C:\]-[사용자]-[사용자 아이디]-[.cursor]-[.skills]-[login-join-skill-builder] 폴더에 스킬 마크다운 파일이 저장된다.

/create-skill

1. Skill name: **login-join-skill-builder**

2. Description

- 다양한 프레임워크에 맞춰 로그인, 회원가입, JWT 인증 로직 및 UI 폼을 규격화하여 생성해주는 보안 인증 스킴이다.
- 이 스킴은 모든 프로젝트에서 사용할 **개인** 스킴이다.

3. Trigger Keywords: 로그인, 회원가입, Login, Join

4. Skill Prompt (AI 행동 지침)

[데이터베이스 요구사항]

1. Prisma와 SQLite를 연동하여 현재 데이터베이스에 'Users' 테이블 스키마를 정의한다.
2. 테이블 필드는 email(이메일), password(비밀번호), nickname(닉네임), createdAt(가입일)로 구성한다.

[회원가입 기능]

1. 이메일과 비밀번호(8자리이상, 영문자, 숫자, 특수문자 반드시 포함), 닉네임(중복체크)을 입력
2. 비밀번호는 암호화(`bcrypt`)하여 데이터베이스에 저장

[로그인 기능]

1. 이메일과 비밀번호를 입력
2. 인증 성공 시 JWT를 사용하여 로그인 상태 유지, 틀리면 정확한 실패 사유 발생

[UI/UX 요구사항]

1. 화면 중앙에 깔끔하고 모던한 카드(Card) 레이아웃 형태로 로그인 폼 배치
2. 로그인 폼 하단에 '아직 회원이 아니신가요? 회원가입' 링크를 두어, 클릭 시 회원가입 폼으로 이동
3. 회원가입 폼 하단에 '이미 계정이 있으신가요?' '로그인' 링크를 두어, 클릭 시 로그인 폼으로 이동

[요구사항]

1. 로그인, 회원가입 폼을 생성하여 네비게이션 바 '로그인' 버튼과 연결한다.
2. 회원가입 성공 시 로그인 페이지로 이동한다.
3. 로그인 성공 시 '로그인' 버튼을 '로그아웃' 버튼으로 변경하고 '로그아웃' 버튼 왼쪽에 로그인한 `닉네임`이나 `이메일`을 출력한다.
4. 로그아웃 버튼 클릭 시 '정말로 로그아웃 하시겠습니까?' 모달창을 띄우고 '예'인 경우 쿠키 내용을 삭제하고 홈페이지로 이동한다.
5. '메모랭', '체크리스트'는 로그인한 사용자만 사용가능하도록 한다. 로그인 성공 시 선택했던 페이지로 이동한다.

7. 구글 로그인 추가

OAuth 2.0는 사용자가 비밀번호를 노출하지 않고도, 구글이나 카카오 같은 대형 서비스가 보증하는 '일회용 출입증(Access Token)'을 받아 간단하게 로그인할 수 있게 해주는 글로벌 표준 기술이다.

- 구글 소셜 로그인 기능을 추가하는 프롬프트로 실행한다.

1. 기존에 만들어진 일반 로그인 폼과 백엔드 인증 로직에 구글 소셜 로그인(OAuth 2.0) 기능을 추가한다.
2. 구글 소셜 유저용 DB 필드 추가와 백엔드 콜백 라우터, 그리고 프론트엔드 버튼을 배치하고 연동 코드를 한 번에 작성한다.

- Google Cloud Console에서 OAuth 2.0 클라이언트 ID를 생성한다.

- 1) [구글 클라우드 콘솔]-[프로젝트]-[새 프로젝트]에서 새 프로젝트(vibe-coding)를 생성한다.
- 2) [vibe-coding]-좌측 메뉴 [API 및 서비스]-[OAuth 등의 화면]-[앱 정보]-[대상: 외부]-[연락처 정보]-[만들기] 클릭
- 3) [vibe-coding]-좌측 메뉴 [API 및 서비스]-[사용자 인증 정보]-[사용자 인증 정보 만들기]-[OAuth 클라이언트 ID]-[애플리케이션 유형]-[만들기] 클릭
- 4) [vibe-coding]-좌측 메뉴 [API 및 서비스]-[사용자 인증 정보] OAuth 2.0 클라이언트 ID와 클라이언트 비밀번호를 복사한다.
- 5) .env 파일에 GOOGLE CLIENT ID, GOOGLE CLIENT SECRET, GOOGLE CALLBACK URL 값들을 입력한다.

[illegible]

GOOGLE_CLIENT_SECRET="xx"

GOOGLE_CALLBACK_URL="http://localhost:3000/api/auth/google/callback"

- 산악회 커뮤니티

로그인(이전 login-join-builder Skill 사용)과 글쓰기가 가능한 산악회 커뮤니티 페이지를 작성하여 데이터베이스와 웹 사이트를 배포한다.

[c:]-[data]-[cursor] 폴더에서 **hiking-community** 폴더를 생성하여 데이터베이스 관련 프로그램들을 작성한다.

1. 메인(홈) 페이지 작성

[<https://getdesign.md/>]-[Airbnb] DESIGN.md 내용을 복사하여 DESIGN.md 파일을 생성해 붙여 넣는다.

목표: 아래 기술스택과 내용을 참고하여 '산들바람' 산악회 커뮤니티 메인 페이지를 작성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 데이터베이스: SQLite + Prisma ORM (데이터베이스 이름: hiking.db)
3. 디자인: 'Tailwind' CDN, 'lucide' CDN
4. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 페이지로 작성한다.
2. 네비게이션 바(Navigation Bar):
 - 좌측에 산 모양 아이콘과 산악회 이름('산들바람'), '산행 일정', '활동 후기', '자유게시판'
 - 우측에 '로그인' 표시
 - 각 페이지는 나중에 제작예정
3. 히어로 섹션(Hero Section):
 - 배경 이미지: 산악회와 어울리는 배경 이미지를 검색하여 링크
 - 메인 카피: '혼자 가면 지지만, 함께 가면 정답입니다.'
 - 서브 카피: '초보 등린이부터 전문 백패커까지, 매주 대한민국 방방곡곡 멋진 명산으로 떠나는 즐거운 등산 커뮤니티'
 - CTA(행동 유도) 버튼: '이번 주 산행 일정 보기'
4. 메인 콘텐츠(Main Contents): 1행 4열 그리드 레이아웃에 활동 후기 4장의 사진과 제목, 간단한 내용을 카드로 출력
5. 푸터(Footer): 간단하게 더미 데이터로 생성

[디자인 요구사항]

1. **@DESIGN.md** (Airbnb) 디자인 마크다운 파일을 참고하여 테마를 설정한다.
2. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
3. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
4. 아이콘은 'lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.

2. 로그인/회원가입 페이지 작성

- 이전에 생성한 스킴([설정]-[Rules, Skills, Subagents]-login-join-builder)을 이용하여 로그인/회원가입 페이지를 작성한다.

[요구사항]

1. 로그인, 회원가입 폼을 생성하여 네비게이션 바 '로그인' 버튼과 연결한다.
2. 회원가입 성공 시 로그인 페이지로 이동한다.
3. 로그인 성공 시 '로그인' 버튼을 '로그아웃' 버튼으로 변경하고 '로그아웃' 버튼 왼쪽에 로그인한 '닉네임'을 출력한다.
4. 로그아웃 버튼 클릭 시 '정말로 로그아웃 하시겠습니까?' 모달창을 띄우고 '예'인 경우 쿠키 내용을 삭제하고 홈페이지로 이동한다.

- 구글 소셜 로그인 기능을 추가하는 프롬프트로 실행한다.

1. 기존에 만들어진 일반 로그인 폼과 백엔드 인증 로직에 구글 소셜 로그인(OAuth 2.0) 기능을 추가한다.
2. 구글 소셜 유저용 DB 필드 추가와 백엔드 콜백 라우터, 그리고 프론트엔드 버튼을 배치하고 연동 코드를 한 번에 작성한다.

- **.env**에 GOOGLE CLIENT ID, GOOGLE CLIENT SECRET, GOOGLE CALLBACK URL 값들을 입력한다.

```
GOOGLE_CLIENT_ID="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"  
GOOGLE_CLIENT_SECRET="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"  
GOOGLE_CALLBACK_URL="http://localhost:3000/api/auth/google/callback"
```

3. 자유 게시판 페이지 작성

/create-skill

1. Skill name: **posts-skill-builder**

2. Description: 다양한 프레임워크에 맞춰 게시물 목록, 등록, 정보, 수정, 삭제(CURD) 작업을 수행하기 위한 모든 프로젝트에서 사용할 **개인** 스킴이다.

3. Trigger Keywords: 게시물, 게시판, Post, board

4. Skill Prompt (AI 행동 지침)

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 'Posts' 테이블 스키마를 정의한다.

2. 테이블 필드: id, writer(작성자:nickname), title(제목), content(내용) createdAt(작성일), updatedAt(수정일)

[게시글 목록 페이지 기능]

1. 최근에 작성된 글 순으로 데스크톱인 경우 표(id, title, writer, createdAt)로, 모바일 경우 카드로 출력한다.

2. 검색어 입력창과 검색 버튼을 왼쪽 상단에 배치하고 검색기능(제목, 내용, 작성자)을 구현한다.

3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:5, visiblePages:10), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

4. 게시물 제목을 클릭하면 게시물 정보 페이지로 이동한다.

5. 게시물 목록 오른쪽 상단에 '글쓰기' 버튼을 배치한다.

- 로그인 전: 로그인 페이지로 이동 후 로그인에 성공하면 게시물 쓰기 페이지로 이동

- 로그인 후: 게시물 쓰기 페이지로 이동

[게시글 쓰기 페이지 기능]

1. 제목, 내용을 입력하여 저장한 후 게시물 목록 페이지로 이동한다.

2. 제목, 내용을 입력하기 전에는 '취소', '등록' 버튼 상태를 Disabled 상태로 유지한다.

[게시글 정보 페이지 기능]

1. 해당 게시판 글 정보를 카드(Card) 레이아웃 형태로 출력한다.

2. 게시물 정보 카드(Card) 내가 쓴 글인 경우에만 오른쪽 상단에 '수정', '삭제' 버튼을 배치한다.

3. 삭제 시 '정말로 게시글을 삭제하실래요?' 모달창을 띄우고 '예'인 경우에만 삭제 후 게시물 목록 페이지로 이동한다.

[게시글 수정 페이지 기능]

1. 수정 내용 저장 시 '정말로 게시글을 수정하실래요?' 모달창을 띄우고 '예'인 경우에만 저장 후 게시물 정보 페이지로 이동한다.

2. 제목, 내용을 수정하기 전에는 '취소', '등록' 버튼 상태를 Disabled 상태로 유지한다.

[UI/UX 요구사항]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 페이지로 작성한다.

2. 게시물 내용 입력은 'Quill' 에디터를 사용한다.

3.작성일, 수정일 출력은 'YYYY-MM-DD HH:mm:ss' 포맷으로 출력한다.

4. 모든 모달창의 헤더는 기본 버튼의 테마 색상과 같도록 설정한다.

[요구사항]

1. 게시물 프로그램을 생성하여 네비게이션 바 '자유 게시판' 메뉴와 연결한다.

2. 게시물 관련 모든 페이지는 메인 콘텐츠 넓이와 같게 디자인한다.

3. 게시물 등록은 로그인한 사용자만 가능하도록 한다.

4. 'Posts' 스키마에 'writer' '초록이'와 '등린이', 제목과 내용을 더미 데이터로 37개를 등록해줘.

4. 산행 일정 페이지 작성

목표: 산행 일정 페이지를 작성하고 네비게이션 바의 '산행 일정' 메뉴와 연결한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 'hiking.db' 데이터베이스에 'Schedules' 테이블 스키마를 정의한다.

2. 테이블 필드:

- id, writer(nickname), title(예: "초보자도 환영하는 북한산 백운대 산행")

- description(상세 내용 및 준비물), max_persons(최대 모집인원)

- scheduled_at(산행 시작 예정 일시),- created_at (일정 생성 일시), updated_at (TIMESTAMP): 일정 수정 일시

[기능 및 UI/UX 요구사항]

- 산행 일정 페이지에 대한 기본 CRUD 기능을 구현한다. (일정 생성, 전체 조회, 상세 조회, 수정, 삭제)

5. 활동 후기 페이지 작성

목표: 활동 후기 페이지를 작성하고 네비게이션 바의 '활동 후기' 메뉴와 연결한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 'hiking.db' 데이터베이스에 'ActivityLog' 테이블 스키마를 정의한다.

2. 테이블 필드:

- id, writer(nickname), title(예: "북한산 백운대 산행 후기"), content(내용), imagePath(사진: nullable 가능), createdAt(작성일)

[기능 및 UI/UX 요구사항]

1. 활동 후기 페이지에 대한 기본 CRUD 기능을 구현한다. (후기 생성, 전체 조회, 상세 조회, 수정, 삭제)

2. 파일 업로드를 위해 `multer` 라이브러리를 사용한다. (업로드 사진은 `public/uploads` 폴더에 저장하고 DB에는 파일명 또는 경로만 저장)

[요구사항]

1. [메인 콘텐츠] - [활동 후기] 아래에 최근 산행 일정 데이터 10개를 추출하여 카드로 출력한다.

- 카드내용: 제목, 내용일부, 산행 일시, 모집인원 출력

- 화면출력(Carousel): 4개의 아이템을 화면에 출력하고 부드럽게 슬라이드 이동을 한다.

2. [메인 콘텐츠] - [산행 일정] 아래에 최근 게시판 게시글 5개를 출력 한다.

6. SQLite 데이터베이스를 Supabase로 마이그레이션(Migration)

슈파베이스는 백엔드 개발 없이 데이터베이스, 로그인, 파일 저장 등을 API 형태로 바로 쓸 수 있게 해주는 오픈소스 백엔드 서비스이다.

• 슈파베이스에 프로젝트 생성

1) <https://supabase.com/> 사이트에서 회원가입(Sign Up)을 하고 로그인(Sign In)한다.

2) [Your Organizations/Start your project]-[New Project]에서 프로젝트 이름(hiking)과 비밀번호(자동생성 후 저장)를 입력한다.

3) 비밀번호를 분실한 경우 [hiking 프로젝트]-[Connect]-[Direct]-[Connect your app]-[Connection string]-[Reset password]로 새로 생성한다.

• MCP를 사용한 슈파베이스와 커서 연결

MCP는 모델 컨텍스트 프로토콜(Model Context Protocol)의 약자이며, AI가 개발자의 컴퓨터 환경(컴파일러, 데이터베이스, 파일 시스템 등) 및 외부 도구와 직접 소통하고 행동할 수 있도록 연결해 주는 표준 통신 규격(브리지)을 뜻한다.

1) [Supabase 로그인]-[프로필 아이콘]-[Account]-[Access Tokens]-[Generate new token] 버튼을 클릭한다.

2) 토큰이름(Name)에 'cursor-mcp'를 입력하고 토큰 문자열을 반드시 따로 복사해 둔다. (창을 닫으면 다시 볼 수 없다)

3) Cursor 프로젝트 루트 디렉터리에 .cursor 폴더를 생성하고 내부에 mcp.json 파일을 만들고 아래 코드를 그대로 붙여 넣는다.

```
{
  "mcpServers": {
    "supabase": {
      "command": "npx",
      "args": [
        "-y", "@supabase/mcp-server-supabase@latest",
        "--access-token", "여기에 1단계에서 복사한 액세스 토큰 입력"
      ]
    }
  }
}
```

4) [Cursor]-[Settings]-[Tools & MCP] 활성화하면 최초 활성화 시 Supabase 브라우저 창이 열리고 권한승인(Authorize)을 하면 연결이 완료된다.

• MCP를 사용한 데이터베이스 마이그레이션

로컬 SQLite 데이터베이스를 클라우드 Supabase의 'hiking' 데이터베이스로 마이그레이션 해줘.

1) [Supabase 사이트]-[hiking 데이터베이스]-[Connect]-[ORM]-[Configure ORM]에서 DATABASE_URL를 복사한다.

2) 복사한 URL을 .env 파일에 등록 후 [YOUR-PASSWORD] 자리에 데이터베이스 비밀번호(XXXX)를 입력한다.

내 소스코드를 로컬 SQLite 대신 클라우드 Supabase의 데이터베이스를 사용하도록 수정해줘.

• 웹 스크래핑과 웹 크롤링

웹 크롤링(Web Crawling)은 자동화 봇이 여러 웹페이지를 체계적으로 탐색해 링크를 타고 다니며 데이터를 수집하는 과정(목차 생성)이고, 웹 스크래핑(Web Scraping)은 특정 웹페이지에서 사용자가 원하는 특정 데이터만을 정밀하게 추출하는 과정(내용 수집)이다.

1. 웹 스크래핑(정적 페이지 데이터 수집)

단순히 HTML 페이지를 가져와서 텍스트나 이미지만 뽑아낼 때 사용한다. 브라우저를 띄우지 않아 속도가 압도적으로 빠르고 메모리가 적게 사용된다. Axios(엑시오스), Cheerio(체리오) 라이브러리를 사용한다.

2. 웹 크롤링(동적 페이지 데이터 수집)

로그인이 필요하거나, 버튼을 눌러야 하거나, 스크롤을 내려야 데이터가 로드되는 자바스크립트 기반 동적 웹사이트에 필수적이다. Cursor에게 스크린샷이나 DOM 구조를 주고 코드를 짜달라고 할 때 가장 잘 작동한다. Playwright, Puppeteer(퍼페티어) 라이브러리를 사용한다.

[c:]-[data]-[cursor] 폴더에서 **crawling** 폴더를 생성하여 데이터 수집을 위한 crawling 프로그램을 작성한다.

1. 메인(홈) 페이지 작성

[https://getdesign.md/]-[Claude] DESIGN.md 내용을 복사하여 DESIGN.md 파일을 생성해 붙여 넣는다.

목표: 아래 기술스택과 내용을 참고하여 '크롤링' 서비스 메인 페이지를 작성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 데이터베이스: SQLite + Prisma ORM (데이터베이스 이름: crawling.db)
3. 디자인: 'Tailwind' CDN, 'Lucide' CDN
4. 서버 구동: `npm run dev`

[메인 페이지 레이아웃]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 필수
2. 네비게이션 바(Navigation Bar): 각 페이지는 나중에 제작예정
- 좌측에 제목 텍스트 로고, 우측에 '공지사항', '고객센터', '로그인'
3. 메인 콘텐츠는 2행 3열 그리드 레이아웃에 제목, 간단한 내용을 카드로 출력
- 그리드 아이템: '거래상위 Top종목', '증권주요뉴스', '할리스매장', '할리스커피매뉴', '네이버뉴스', '현재날씨'
4. 푸터(Footer): 간단하게 데미 데이터로 생성

[디자인 요구사항]

1. @DESIGN.md (Claude) 디자인 마크다운 파일을 참고하여 테마를 설정한다.
2. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
3. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
4. 아이콘은 'Lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.
5. 글꼴은 구글 폰트 https://fonts.google.com/?lang=ko_Kore에서 'IBM Plex Sans KR'로 사용한다.

2. Axios, Cheerio 라이브러리를 이용한 데이터 수집

목표: 메인 콘텐츠 '거래상위 Top종목'을 클릭하면 요구 사항을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 기술스택: 'Axios', 'Cheerio' 라이브러리
2. 타겟 URL: `https://finance.naver.com/`
3. 수집데이터: 'Top종목'-'NXT'-'거래상위' Top100종목
4. 한글이 깨지면 'iconv-lite'를 사용하여 HTML 데이터를 한국어(EUC-KR)로 디코딩한다.

[UI/UX 요구사항]

1. 추출된 텍스트들은 순번과 함께 테이블로 출력한다.
2. 데이터 수집중일 때는 스피너를 출력한다.
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:5, visiblePages:10), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.
4. '다시수집' 버튼을 이 페이지 서브 제목과 같은 행 오른쪽 끝에 배치한다.
5. 이 페이지('거래상위 Top종목')에서만 테이블안의 글자 크기를 1.1em으로 출력한다.
6. 전일비 상승, 하락은 삼각형, 역삼각형을 출력한다.

목표: 메인 콘텐츠 '증권주요뉴스'를 클릭하면 아래 내용을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 타겟 URL: `https://finance.naver.com/`
2. 수집데이터: '주요뉴스'의 제목, 내용, 기사링크 수집(Top4 뉴스)

[UI/UX 요구사항]

1. 추출된 텍스트들은 순번과 함께 그리드 2행 2열 레이아웃에 출력한다.
2. 데이터 수집중일 때는 스피너를 출력한다.
3. 페이지 넓이는 메인 콘텐츠 넓이와 같게 한다.
4. '다시수집' 버튼을 이 페이지 서브 제목과 같은 행 오른쪽 끝에 배치한다.

목표: 메인 콘텐츠 '홀리스매장'을 클릭하면 아래 내용을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 타겟 URL: `https://www.hollys.co.kr/store/korea/korStore2.do`
2. 수집데이터: 지역, 매장명, 현황, 주소, 매장 서비스, 전화번호 (전체 페이지)

[UI/UX 요구사항]

1. 추출된 텍스트들은 순번과 함께 테이블로 출력한다.
2. 데이터 수집중일 때는 스피너를 출력한다.
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:5, visiblePages:1-), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

목표: 메인 콘텐츠 '홀리스커피메뉴'를 클릭하면 아래 내용을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 타겟 URL: `https://www.hollys.co.kr/menu/espresso.do`
2. 수집데이터: 메뉴이미지, 메뉴이름

[UI/UX 요구사항]

1. 추출된 텍스트들은 2행 5열로 그리드 레이아웃에 출력한다.
2. 수집 데이터가 10개 이상인 경우 'More' 등근 버튼을 마지막 데이터인 경우에는 'Close' 등근 버튼을 출력한다.

목표: 메인 콘텐츠 '네이버뉴스'를 클릭하면 아래 내용을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 타겟 URL: `https://news.naver.com/`
2. 수집데이터: '정치', '경제', '사회', '생활/문화', 'IT/과학' 섹션별 썸네일 이미지, 제목, 내용, 출처, 링크 (상위 3개만 추출)

[UI/UX 요구사항]

1. 추출된 텍스트들은 섹션별 탭을 만들고 이미지, 제목, 내용(한줄 만 출력하고 생략), 출처를 출력한다.
2. 데이터 수집중일 때는 스피너를 출력한다.
3. 페이지 넓이는 메인 콘텐츠 넓이와 같게 한다.
4. '다시수집' 버튼을 이 페이지 서브 제목과 같은 행 오른쪽 끝에 배치한다.

목표: 메인 콘텐츠 '현재날씨'를 클릭하면 아래 내용을 참고하여 크롤링한다.

[데이터수집 요구사항]

1. 타겟 URL: `https://weather.naver.com/`
2. 수집데이터:
 - 현재 위치 정보(네이버 날씨 페이지 기본 위치): 지역이름, 현재 기온, 최저 기온, 최고 기온, 현재 날씨
 - 지도상의 전국 날씨 정보: 지역이름, 현재 기온, 현재 날씨

[UI/UX 요구사항]

1. 현재 위치 정보는 카드 안에 출력한다.
2. 지도상의 전국 날씨 정보는 한 화면에 8개 카드, 카드 1개씩 좌/우 이동하는 캐러셀(자동 플레이)기능을 추가한다.
3. 날씨는 어울리는 아이콘으로 출력한다. (예: 맑음/해, 흐림/구름)
4. 데이터 수집중일 때는 스피너를 출력한다.

• 카카오 검색, 지도 API

카카오 검색 API는 Daum 포털 검색 데이터를 기반으로 책, 블로그, 카페, 동영상 등 다양한 콘텐츠를 개발자가 자신의 서비스에 연동할 수 있게 해주는 REST API 인터페이스이다. REST API 방식(JSON 형태 응답)을 통해 웹/앱에서 쉽고 빠르게 검색 기능을 구현할 수 있으며, 일일 쿼터 내 무료로 이용 가능하다. **카카오 지도 API(Kakao Maps API)**는 카카오가 제공하는 지도 데이터와 인프라를 외부 개발자가 자신의 웹사이트나 모바일 앱에 무료로 가져와 사용할 수 있도록 제공하는 소프트웨어 개발 도구(SDK/API)이다.

[c:]-[data]-[cursor] 폴더에서 **kakao-api** 프로젝트를 생성하여 카카오 검색 API를 이용한 각종 검색 프로그램을 작성한다.

1. 메인(홈) 페이지 작성

[https://getdesign.md/]-[Apple] DESIGN.md 내용을 복사하여 DESIGN.md 파일을 생성해 붙여 넣는다.

목표: 아래 내용을 참고하여 '카카오 검색' 웹 사이트 레이아웃을 구성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 데이터베이스: SQLite + Prisma ORM (데이터베이스 이름: kakao.db)
3. 디자인: 'Tailwind' CDN, 'Lucide' CDN
4. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 네비게이션 바 (Navigation Bar)
 - 좌측에 로고(어울리는 아이콘), 사이트 제목(조금 크게 눈에 띄게)
 - 우측에 '공지사항', '고객센터', '로그인' (각 페이지는 나중에 제작 예정)
2. 메인 콘텐츠는 1행 5열 그리드 레이아웃에 제목, 간단한 내용을 카드로 출력
 - 그리드 아이템: '도서 검색', '블로그 검색', '동영상 검색', '이미지 검색', '지역검색'
3. 푸터(Footer)
 - 주소, 전화번호, 저작권을 간단한 터미 데이터로 생성하여 출력

[디자인 요구사항]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 페이지로 작성한다.
2. @DESIGN.md (Claude) 디자인 마크다운 파일을 참고하여 테마를 설정한다.
3. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
4. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
5. 모든 페이지에서 작업이 오래 걸릴 때는 스피너(Spinner)와 로딩 메시지 출력한다.
6. 아이콘은 'Lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.
7. 글꼴은 구글 폰트 https://fonts.google.com/?lang=ko_Kore에서 'IBM Plex Sans KR'로 사용한다.

2. 카카오 도서 검색 페이지 작성

카카오 검색 API를 사용하기 위해서 카카오 개발자 센터에서 새로운 앱을 생성한 후 REST API키를 복사하여 .env 파일에 등록한다.

- 1) [카카오 개발자 센터]-[로그인]-[앱]-[앱 생성]에서 새로운 앱을 생성한다.
- 2) [생성한 앱]-[앱]-[플랫폼 키]-[REST API키]를 복사하여 사용한다.
- 3) 프로젝트 .env 파일에 KAKAO_REST_API_KEY="xxxxxxxxxxxxxxxxxxxxxx"를 등록한다.

목표: 아내 내용을 참고하여 도서 검색 페이지를 작성하고 메인 콘텐츠 '도서 검색'을 클릭하면 해당 페이지로 이동한다.

[카카오 공식 문서]

- @Docs <https://developers.kakao.com/docs/ko>
- 도서 검색 REST API

[기능 및 요구사항]

1. 특정 키워드를 입력받아 검색 결과를 출력한다.
2. .env 파일에 카카오 검색을 위한 KAKAO_REST_API_KEY를 참고한다.

[UI/UX 디자인 요구사항]

1. 검색어 입력창은 서브 제목과 같은 행 오른쪽에 배치한다.
2. 검색 결과는 1행 6열(반응형 적용)의 그리드 레이아웃으로 출력한다. (그리드 항목: 카드 형태로 이미지, 제목, 가격 출력)
3. 도서이미지를 클릭하면 도서상세 정보를 Large '모달창'으로 출력한다.
4. 페이지네이션은 데스크톱인 경우 (itemsPerPage:6, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

3. 카카오 블로그 검색 페이지 작성

목표: 안내 내용을 참고하여 블로그 검색 페이지를 작성하고 메인 콘텐츠 '블로그 검색'을 클릭하면 해당 페이지로 이동한다.

[카카오 공식 문서]

- @Docs <https://developers.kakao.com/docs/ko>

- 블로그 검색 REST API

[기능 및 요구사항]

- 특정 키워드를 입력받아 검색 결과를 출력한다.

[UI/UX 디자인 요구사항]

1. 검색어 입력창은 서브 제목과 같은 행 오른쪽에 배치한다.
2. 검색 결과를 메인 콘텐츠 넓이와 같은 카드로 출력한다.
 - 카드 안에는 이미지, 블로그명, 일부 요약내용, 작성일, 링크를 보기 좋게 배치하여 출력한다.
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:2, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

4. 카카오 동영상 검색 페이지 작성

목표: 안내 내용을 참고하여 동영상 검색 페이지를 작성하고 메인 콘텐츠 '동영상 검색'을 클릭하면 해당 페이지로 이동한다.

[카카오 공식 문서]

- @Docs <https://developers.kakao.com/docs/ko>

- 동영상 검색 REST API

[기능 및 요구사항]

- 특정 키워드를 입력받아 검색 결과를 출력한다.

[UI/UX 디자인 요구사항]

1. 검색 결과는 2행 5열(반응형 적용)의 그리드 레이아웃으로 카드 안에 들어갈 내용은 아래와 같다.
 - 상단에는 동영상 썸네일을 배치
 - 썸네일 우측 하단에 '재생 시간'을 오버레이(겹침) 형태로 작게 표시하여 배치
 - 썸네일 아래에는 동영상 제목(한줄)과 작성자를 깔끔하게 배치
2. 카드를 클릭하면 해당 동영상의 원래 링크(url)가 새 탭에 열린다.
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:10, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

5. 카카오 이미지 검색 페이지 작성

목표: 안내 내용을 참고하여 이미지 검색 페이지를 작성하고 메인 콘텐츠 '이미지 검색'을 클릭하면 해당 페이지로 이동한다.

[카카오 공식 문서]

- @Docs <https://developers.kakao.com/docs/ko>

- 이미지 검색 REST API

[기능 및 요구사항]

- 특정 키워드를 입력받아 검색 결과를 출력한다.

[UI/UX 디자인 요구사항]

1. 검색 결과는 2행 5열(반응형 적용)의 그리드 레이아웃으로 카드 안에 들어갈 내용은 아래와 같다.
 - 상단에는 썸네일 이미지를 꼭 차게 배치하고 이미지 크기가 제각각이더라도 카드 형태가 깨지지 않도록 고정 비율을 유지
 - 이미지 아래에는 출처명('display_sitenam')을 깔끔하게 배치
2. 썸네일 이미지를 클릭하면 해당 이미지의 원본 파일이 중간 사이즈 모달창에 출력되도록 구현한다.
 - 모달 안 원본 이미지는 위를 기준으로 꼭 차게 배치하고 이미지 크기가 제각각이더라도 카드 형태가 깨지지 않도록 고정 비율을 유지
 - 모달 푸터 왼쪽에 출처명(강조)과 이미지크기를 출력하고 푸터 오른쪽에 '원본보기->' 링크 텍스트를 배치
 - 원본 이미지가 없거나 오류가 발행하는 경우 더미 이미지를 출력
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:10, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

9. '즐거찾기' 관리

목표: 도서 검색 페이지에서 사용자(이메일)별로 '즐거찾기'에 도서 정보를 저장, 삭제하는 프로그램을 작성한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 'kakao.db' 데이터베이스에 'BookFavorites' 테이블 스키마를 정의한다.
2. 테이블 필드: id, email, 도서 정보의 모든 컬럼

[기능 및 요구사항]

1. 로그인 전 기능
 - 빈 하트를 클릭하면 로그인 페이지로 이동하여 로그인 후 도서 검색 페이지로 돌아온다.
2. 로그인 후 기능
 - 로그인한 사용자(이메일)의 '즐거찾기'에 저장된 도서이면 배경색을 채운하트로 출력하고 없는 도서이면 배경색이 빈 하트를 출력한다.
 - 배경색이 빈 하트를 클릭하면 도서정보를 데이터베이스에 저장한다.
 - 배경색이 채워진 하트를 클릭하면 도서정보를 데이터베이스에서 삭제한다.

[UI/UX 디자인 요구사항]

1. 하트 아이콘은 도서 썸네일 이미지 오른쪽 상단에 이미지와 약간 겹쳐서 분홍색으로 출력한다.
2. 도서가 저장, 삭제될 때 마다 토스트에 '즐거찾기에 저장되었습니다.', '즐거찾기에서 삭제되었습니다.' 메시지를 출력한다.
3. 'Favorites'에 데이터가 변경될 때마다 '도서 검색' 페이지 큰제목 옆에 실시간으로 배지('하트'+변경된 도서 숫자만)를 출력한다.

목표: '즐거찾기'에 저장된 데이터를 관리(목록, 삭제)하는 페이지를 작성한다.

[데이터베이스 스키마]: 도서정보 즐겨찾기('Favorites')

[기능 및 요구사항]

1. '즐거찾기'에 저장된 최근 데이터 순으로 링크를 포함해 2열의 그리드 레이아웃으로 깔끔하게 배치하여 출력한다.
2. 카드 안에 '삭제' 아이콘을 배치하고 클릭 시 '즐거찾기' 데이터베이스에서 도서정보를 삭제한다.
3. 전체 선택, 부분 선택하여 '즐거찾기' 데이터베이스에서 도서정보를 삭제하는 기능을 추가한다.

[UI/UX 디자인 요구사항]

1. 페이지 넓이를 메인 콘텐츠 넓이와 같게 한다.
2. '하트'+변경된 도서 수(숫자만) 클릭 시 '즐거찾기' 관리 페이지로 이동한다.
3. '즐거찾기' 도서정보 삭제 시 모달창에 '정말로 삭제하시겠습니까?'를 물어보고 '예'인 경우만 삭제한다.
4. 페이지네이션은 데스크톱인 경우 (itemsPerPage:4, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.

목표: 블로그 검색 페이지에서 사용자(이메일)별로 '즐거찾기'에 블로그 정보를 저장, 삭제하는 프로그램을 작성한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 'kakao.db' 데이터베이스에 'BlogFavorites' 테이블 스키마를 정의한다.
2. 테이블 필드: id, email, 블로그 정보의 모든 컬럼

[기능 및 요구사항]

1. '블로그 검색' 페이지에도 '도서 검색' 페이지의 '즐거찾기'에 저장, 삭제하는 동일한 기능을 추가한다.
2. '즐거찾기'에 저장된 데이터를 관리(목록, 삭제)하는 페이지를 도서와 동일하게 작성한다.

10. SQLite 데이터베이스를 Supabase로 마이그레이션(Migration)

- 1) [Supabase 사이트]-[New Project]에서 프로젝트 이름(kakao)과 비밀번호(자동생성 후 저장)를 입력한다.

로컬 SQLite 데이터베이스를 클라우드 Supabase의 'kakao' 데이터베이스로 Migration 해줘.

- 2) [Supabase 사이트]-[kakao 데이터베이스]-[Connect]-[ORM]-[Configure ORM]에서 DATABASE_URL를 복사한다.
- 3) 복사한 URL을 .env 파일에 등록 후 [YOUR-PASSWORD] 자리에 데이터베이스 비밀번호(xxxx)를 입력한다.

내 소스코드를 로컬 SQLite 대신 클라우드 Supabase 데이터베이스를 사용하도록 수정해줘.

11. Vercel에 배포

- 1) vercel에 작성한 프로젝트를 배포(vercel --prod)하고 카카오 지도를 vercel에서도 사용하기 위해 생성된 프로젝트 도메인을 복사한다.
- 2) [생성한 앱]-[플랫폼 키]-[Default JavaScript KEY]-[JavaScript SDK 도메인]에 위에서 복사한 도메인을 추가 등록한다.

- **OpenAI API**

OpenAI API는 ChatGPT를 구동하는 강력한 인공지능 모델을 개발자가 자신의 앱이나 웹사이트에 직접 연동할 수 있게 해주는 인터페이스이다. 복잡한 머신러닝 구축 없이, 코딩 몇 줄만으로 자연어 처리, 이미지 생성 등의 기능을 서비스에 추가할 수 있다. 월 구독제인 ChatGPT Plus와 달리, API는 사용량 기반 요금제로 운영된다. 텍스트 입력과 출력 단위인 '토큰(Token)'의 양에 따라 비용이 청구된다.

[c:]-[data]-[cursor] 폴더에서 **open-api** 프로젝트 폴더를 생성하여 카카오 검색 API를 이용한 각종 검색 프로그램을 작성한다.

1. 메인(홈) 페이지 작성

[<https://getdesign.md/>]-[NVIDIA] DESIGN.md 내용을 복사하여 DESIGN.md 파일을 생성해 붙여 넣는다.

목표: 아래 내용을 참고하여 '오픈AI-API' 웹 사이트 레이아웃을 구성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 디자인: 'Tailwind' CDN, 'Lucide' CDN
3. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 네비게이션 바 (Navigation Bar)
 - 좌측에 로고(어울리는 아이콘), 사이트 제목(조금 크게 눈에 띄게)
 - 우측에 '공지사항', '고객센터', '로그인' (각 페이지는 나중에 제작 예정)
2. 메인 콘텐츠는 1행 4열 그리드 레이아웃에 제목, 간단한 내용을 카드로 출력
 - 그리드 아이템: '텍스트 요약', '이메일 생성', '데이터분석과 시각화', '챗봇 구현'
3. 푸터(Footer)
 - 주소, 전화번호, 저작권을 간단히 더미 데이터로 생성하여 출력(여백 줄여서)

[디자인 요구사항]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 페이지로 작성한다.
2. @DESIGN.md (Claude) 디자인 마크다운 파일을 참고하여 테마를 설정한다. (Light/Dark Mode)
3. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
4. 카드(박스)에 마우스를 올리면 살짝 떠오르는 부드러운 애니메이션 효과와 고급스러운 그림자를 추가한다.
5. 모든 페이지에서 작업이 오래 걸릴 때는 스피너(Spinner)와 로딩 메시지 출력한다.
6. 아이콘은 'lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.
7. 글꼴은 구글 폰트 'https://fonts.google.com/?lang=ko Kore'에서 'IBM Plex Sans KR'로 사용한다.

2. OpenAI API 텍스트 요약

OpenAI API의 텍스트 요약 기능은 GPT같은 거대 언어 모델(LLM)을 활용하여, 방대한 양의 텍스트 데이터에서 핵심 내용만 추출하거나 사용자가 원하는 스타일로 압축해주는 기능을 말한다. 단순히 문장을 자르는 것이 아니라, 문맥을 이해하고 새로운 문장으로 재구성한다.

- 1) 'https://openai.com/ko-KR/index/openai-api/' 사이트로 이동 후 [API 플랫폼]으로 로그인하여 API Key를 생성한다.
- 2) .env 파일에 OPENAI_API_KEY 변수에 생성한 API Key 값을 등록한다.

[illegible]

목표: 아래 기능을 만족하는 페이지를 작성하고 메인 콘텐츠 '텍스트 요약'을 클릭하면 해당 페이지로 이동한다.

[기술스택]: `Axios`, `Cheerio` 라이브러리

[기능 및 요구사항]

1. 타겟 URL: `https://news.naver.com/section/105` [IT/과학] 섹션으로 이동
2. 수집데이터:
 - 기사 상위 2개 기사를 수집(연론사명, 기사 제목, 기사 내용)하여 1열의 그리드 레이아웃에 출력한다.
 - 기사 내용으로는 상세보기의 본문내용만 수집하고 3줄/전체 보기를 토글, 제목 클릭 시 원문 URL로 이동
 - 한글이 깨지면 `iconv-lite`를 사용하여 HTML 데이터를 한국어(EUC-KR)로 디코딩한다.

[UI/UX 디자인 요구사항]

1. 데이터 수집이 완료될 때 까지 로딩 스피너를 출력한다.
2. 페이지 넓이를 매인 콘텐츠와 같도록 출력한다.
3. 그리드 아이템 우측 상단에 '기사요약'(아이콘 포함) 버튼을 기능 없이 배치만 한다.

목표: 텍스트 요약 페이지에서 '기사요약' 버튼을 클릭하면 OpenAI API를 이용해 기사를 요약하는 기능을 추가한다.

[기술스택]

1. 라이브러리: 공식 OpenAI 패키지 최신 버전 사용
2. API 키 관리: .env 파일에서 OPENAI_API_KEY 사용
3. OPENAI gpt-4o-mini 모델을 사용하여 토큰 절약

[기능 및 요구사항]

1. 요약 1회 호출시 한글/영어 결과를 함께 받아 Toggle하면 즉시 전환한다. (API 비용 1회)
2. 한 문장의 핵심 한 줄 요약을 최상단에 배치한다.
3. 기사의 핵심 내용을 3가지 항목의 글머리 기호(bullet points)로 요약한다.
4. 각 글머리 기호는 20단어 내외의 짧고 간결한 문장으로 작성한다.
5. 전문 용어나 어려운 단어는 쉬운 언어로 바꾸어 설명한다.
6. 기사에 등장하는 주요 수치나 날짜, 고유명사는 반드시 포함한다.
7. 주관적인 의견이나 추측은 배제하고, 오직 본문의 사실만을 바탕으로 작성한다.

[UI/UX 디자인 요구사항]

1. 요약 결과는 모달창(중간 사이즈)에 출력한다.
2. 한글/영어 Toggle 버튼은 모달창 헤더 영역 오른쪽 상단에 배치한다.
3. API키가 없거나 호출에 실패했을 때 모달창(헤더: 버튼 색', 본문: 에러 메시지)을 출력한다.

2. OpenAI API 이메일 생성

OpenAI API 이메일 생성이란, OpenAI에서 제공하는 인공지능 모델을 프로그램에 연결하여 사용자의 요구사항에 맞는 이메일 초안을 자동으로 작성해 주는 기술을 의미한다. 개발자나 기업이 자신들의 시스템 내에 '메일 자동 작성 기능'을 직접 구현할 때 핵심적으로 사용된다.

목표: 아래 기능을 만족하는 '이메일 생성' 페이지를 작성하고 메인 콘텐츠 '이메일 생성' 클릭하면 해당 페이지로 이동한다.

[기능 및 요구사항]

1. 입력 데이터: 수신자 이름, 핵심용건(예: '다음 주 화요일 오후 3시 프로젝트 Kickoff 미팅 제안'), 이메일 톤앤매너
2. '이메일 생성' 버튼을 클릭하면 입력한 데이터를 주입하여 이메일 작성을 요청한다.
3. API로부터 응답(제목과 본문)을 받으면 생성된 이메일을 깔끔하게 출력한다.

[UI/UX 디자인 요구사항]

1. 2열의 그리드 레이아웃으로 화면을 분할한다.
2. 1열에는 입력 폼을 배치하고, 이메일 톤앤매너는 선택박스(비즈니스 표준형, 격식형, 친근형, 확신/요청형, 사과/양해형)로 입력한다.
 - 핵심용건 초기 값으로 '다음 주 화요일 오후 3시 프로젝트 Kickoff 미팅 제안'을 입력한다.
 - 입력 폼 아래에 'AI에게 이메일 생성 요청'(아이콘 포함) 버튼을 배치한다.
3. 2열에는 이메일 생성 결과를 배치한다.
 - 이메일 생성 결과 아래에 '생성된 메일 저장'(아이콘 포함) 버튼을 기능 없이 배치만 한다.

목표: '이메일 생성' 페이지에서 작성한 메일을 데이터베이스에 저장하고 관리하는 페이지를 작성한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 'openai.db' 데이터베이스에 'Emails' 테이블 스키마를 정의한다.
2. 테이블 필드: id, 수신자이름, 핵심용건, 톤앤매너, 생성된 이메일, 저장일시

[기능 및 요구사항]

1. 메일 저장: '생성된 메일 저장' 버튼을 클릭하면 생성된 메일을 데이터베이스에 저장한 후 토스트로 메시지를 출력한다.
2. 저장 메일 목록: 저장한 이메일을 2열 그리드 레이아웃(생성된 이메일 제외)으로 출력한다.(저장일시 역순으로)
3. 저장 메일 수정: 수정 페이지에서는 생성된 이메일만 출력하고 수정 가능하도록 한다.
4. 저장 메일 삭제: 목록 페이지 또는 수정 페이지에서 삭제할 수 있도록 한다.

[UI/UX 디자인 요구사항]

1. 이메일 생성 페이지 부제목과 같은 행 오른쪽 끝에 '저장된 메일 확인' 텍스트 링크를 배치하고 클릭 시 메일 목록 페이지로 이동한다.
2. 메일 목록 페이지 부제목과 같은 행 오른쪽 끝에 '이메일 생성' 텍스트 링크를 배치하고 클릭 시 이메일 생성 페이지로 이동한다.
3. 페이지네이션은 데스크톱인 경우 (itemsPerPage:4, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.
4. 저장된 메일을 수정, 삭제할 경우 모달창을 띄우고 '예'를 선택한 경우에만 수정, 삭제 작업을 한다.
5. 메일 수정 페이지의 넓이는 메인 콘텐츠 넓이와 같게 설정한다.

3. OpenAI API 데이터 분석과 시각화

OpenAI의 API를 데이터 분석에 활용하면, 단순히 수치를 계산하는 것을 넘어 데이터의 의미를 해석하고 인사이트(Insight)를 도출하는 데 탁월한 성능을 발휘한다. 따라서 복잡한 숫자들 사이의 상관관계를 찾아내고 전문적인 보고서 문체로 요약하고 시각화 할 수 있다.

목표: 아래 기능을 만족하는 '데이터분석과 시각화' 페이지를 작성하고 메인 콘텐츠 '데이터분석과 시각화' 클릭하면 해당 페이지로 이동한다.

[기술스택]

-구글 차트(<https://developers.google.com/chart?hl=ko>) 우선 사용.

[기능 및 요구사항]

1. 분석할 데이터파일과 사용자 분석 요구사항을 입력받아 텍스트 분석결과와 시각화 결과를 출력한다.
2. OpenAI가 아래와 같은 JSON 구조로만 응답하도록 시스템 프롬프트를 설정한다.
 - summary: 데이터에 대한 전반적인 요약 내용 (문자열)
 - insights: 요구사항에 맞게 분석된 핵심 인사이트 목록 (배열)
 - chartType: 시각화에 적합한 차트 유형 (구글 차트 주요 유형 전부 사용, horizontalBar: 항목명이 길거나 항목이 많을 때)
 - chartOptions: 차트의 다양한 옵션
 - chartData: 바로 사용할 수 있는 형태의 JSON 데이터 배열

[UI/UX 디자인 요구사항]

1. 2열(1:3 비율)의 그리드 레이아웃으로 화면을 분할한다.
2. 그리드 왼쪽(입력영역)
 - CSV 또는 JSON 데이터 파일을 업로드 할 수 있는 드롭존(Dropzone) 영역
 - 사용자가 분석 요구사항을 자유롭게 텍스트로 입력할 수 있는 텍스트(Textarea) 영역
 - 분석 시작' 버튼 및 로딩 애니메이션 (스피너)
3. 그리드 오른쪽(결과영역)
 - AI의 답변(텍스트)
 - 시각화 결과(간단한 HTML/CSS 기반의 차트 형태)

4. OpenAI API로 챗봇 구현

OpenAI API 챗봇은 OpenAI가 제공하는 강력한 거대언어모델(LLM)을 개발자의 서비스나 애플리케이션에 연동하여 사람처럼 자연스럽게 대화할 수 있도록 만든 인공지능 챗봇이다. 이를 활용하면 서비스 목적에 맞는 맞춤형 AI 비서를 직접 구현할 수 있다.

목표: 아래 기능을 만족하는 '챗봇 구현' 페이지를 작성하고 메인 콘텐츠 '챗봇 구현' 클릭하면 해당 페이지로 이동한다.

[기능 및 요구사항]

1. 이전 대화 흐름을 기억할 수 있도록, 클라이언트로부터 메시지 기록 배열(messages) 전체를 받아와 OpenAI API에 그대로 전달한다.
2. 사용자가 전송을 누르면 즉시 화면에 내 메시지를 추가하고, API 호출이 완료되면 AI의 답변을 대화 내역에 누적한다.

[UI/UX 디자인 요구사항]

1. 메인 콘텐츠 넓이와 같은 깔끔한 대화창 UI를 만든다.
2. 왼쪽에는 '새 대화 시작' 버튼이 있는 심플한 사이드바, 오른쪽에는 메인 채팅창 영역을 배치한다. (3:8)
3. 채팅창 영역 레이아웃
 - 채팅창 하단에는 메시지를 입력하는 창(Input)과 전송 버튼(아이콘)을 배치한다.
 - 사용자가 보낸 메시지는 오른쪽(파란색/보라색 말풍선)에 배치한다.
 - AI가 보낸 메시지는 왼쪽(회색/어두운 배경 말풍선)에 구분되어 배치한다.
4. 메시지가 길어지면 채팅 영역에 스크롤이 생기고, 새 메시지가 오면 자동으로 스크롤이 가장 아래로 내려가도록 처리한다.
5. AI가 답변을 생성 중일 때는 챗봇 이름 옆에 아이콘 + '답변 작성 중...' 또는 로딩 애니메이션을 보여준다.

목표: '챗봇 구현' 페이지에서 대화 내용을 저장, 출력, 삭제하는 기능을 추가한다.

[기능 및 요구 사항]

1. '새 대화 시작' 버튼 클릭 시 첫 번째 메시지가 입력되면 이때부터의 대화내용이 저장일시와 함께 브라우저 로컬스토리지에 된다.
2. 대화 목록 아이템을 클릭하면 채팅창에 대화 내용을 출력한다.
3. 대화 목록 옵션 메뉴의 '삭제' 메뉴를 클릭하면 대화 내용을 삭제한다.

[UX/UI 디자인 요구사항]

1. 좌측 사이드바 (Sidebar)에 대화 목록(저장일시 포함제목)을 출력한다.
2. 채팅창 사용자가 보낸 메시지 말풍선 아래 저장일시를 출력한다.
3. 대화 목록의 각 대화 항목 우측에 옵션 메뉴(점 세 개 아이콘) 배치 -> 클릭 시 '삭제' 메뉴를 표시한다.

● 학사관리 시스템

학사관리를 위한 데이터베이스 스키마를 생성하고 관리자, 사용자(학생), 사용자(교수)별로 아래 기능을 수행하는 프로그램을 작성한다.

- 관리자 모드: 교수관리(등록, 정보수정, 삭제), 학생관리(등록, 정보수정, 삭제), 강좌관리(등록, 정보수정, 삭제), 수강신청관리(조회)
- 사용자(학생) 모드: 학생정보 수정, 수강 신청/취소
- 사용자(교수) 모드: 교수정보 수정, 지도학생 목록, 담당과목 목록, 담당과목별 점수 등록

[c:]-[data]-[cursor] 폴더에서 **learning-management** 프로젝트 폴더를 생성하여 학사관리를 위한 프로그램을 작성한다.

1. 데이터베이스 스키마 생성

목표: 학사관리에서 사용할 데이터베이스 스키마를 아래 조건을 참고해서 실행 전에 먼저 계획서(plan/SCHEMA_PLAN.md)로 작성한다.

[데이터베이스 요구사항]

1. SQLite + Prisma ORM필수
2. 데이터베이스 이름: 'learning'

[데이터베이스 스키마 구성]

1. 교수 테이블(professors)
 - 교수번호: 반드시 문자 4자리, 기본키(PK)
 - 교수이름: 문자 30자리 이하, NOT NULL
 - 교수학과: 문자 30자리 이하, NOT NULL (컴퓨터정보 공학과, 전자전기 공학과, 건축 공학과)
 - 임용일: 날짜
 - 직급: 문자 20자리 이하 (정교수, 부교수, 조교수, 겸임교수, 시간강사)
2. 학생 테이블(students)
 - 학생번호: 반드시 문자 8자리, 기본키(PK)
 - 학생이름: 문자 30자리 이하, NOT NULL
 - 학생학과: 문자 20자리 이하, NOT NULL (컴퓨터정보 공학과, 전자전기 공학과, 건축 공학과)
 - 생년월일: 날짜
 - 지도교수번호: 문자 4자리, 참조키(FK: 교수 테이블 교수번호 참조)
3. 강좌 테이블(courses)
 - 강좌번호: 반드시 문자 3자리, 기본키(PK)
 - 강좌이름: 문자 100자리 이하, NOT NULL
 - 강역시수: 숫자, DEFAULT 0
 - 현재수강인원: 숫자, DEFAULT 0
 - 최대수강인원: 숫자, 최대인원(5)
 - 담당교수번호: 반드시 문자 4자리, 참조키(FK: 교수 테이블 교수번호 참조)
4. 수강신청 테이블(enrollments)
 - 번호: 숫자 기본키(PK), 자동증가
 - 강좌번호: 반드시 문자 3자리, NOT NULL, 참조키(FK: 강좌테이블 강좌번호 참조)
 - 학생번호: 반드시 문자 8자리, NOT NULL, 참조키(FK: 학생테이블 학생번호 참조)
 - 수강신청일: 날짜, DEFAULT 오늘날짜
 - 점수: 숫자, DEFAULT 0
5. 사용자 테이블(users)
 - 아이디: 문자 30글자 이하, 기본키(PK)
 - 비밀번호: 문자 200글자 이하, 반드시 암호화(`bcrypt`)
 - 롤: 숫자, (0:관리자, 1:교수, 2:학생)
 - 가입일: 날짜, DEFAULT 오늘날짜
 - 학생번호: 문자 8글자, 참조키(FK: 학생테이블 학생번호 참조)
 - 교수번호: 문자 4자리, 참조키(FK: 교수테이블 학생번호 참조)

[추가 요구사항]

1. 교수 테이블에 데이터 5개, 학생 테이블 데이터 10개, 강좌 테이블 데이터 5개, 수강신청 테이블 데이터 30개를 더미 데이터로 생성한다.
2. 사용자 테이블에 데이터 5개를 아래 내용을 참고하여 더미 데이터로 생성한다.
 - 관리자 1명: (아이디:'admin')
 - 교수 2명: (아이디:'stu01', 'stu02', 'stu03')
 - 학생 2명: (아이디:'pro01', 'pro02', 'pro03')
 - 비밀번호는 모두 동일하게 'password123!'로 등록한다.

2. 메인(홈) 페이지 작성

- [<https://getdesign.md/>]-[Starbucks] DESIGN.md 내용을 복사하여 DESIGN.md 파일을 생성해 붙여 넣는다.

목표: 아래 내용을 참고하여 '스마트' 캠퍼스 웹 사이트 레이아웃을 구성한다.

[기술스택]

1. 웹 서버: Express + EJS
2. 디자인: 'Tailwind' CDN, 'Lucide' CDN
3. 서버 구동: `npm run dev`

[메인페이지 레이아웃]

1. 네비게이션 바 (Navigation Bar)
 - 좌측에 로고(어울리는 아이콘), 사이트 제목(조금 크게 눈에 띄게)
 - 우측에 '로그인' 메뉴 (각 페이지는 나중에 제작 예정)
2. 히어로 섹션(Hero Section)
 - 학사관리 시스템과 어울리는 이미지를 검색하여 CDN URL로 히어로 이미지(가로 세로 비율을 6:1)를 예쁘고 깔끔하게 출력
 - 제목('스마트 캠퍼스와 함께하는 학사관리')과 간단한 설명, [공지사항] 버튼을 히어로 이미지와 겹치도록 배경을 투명하게 출력
2. 메인 콘텐츠: 우리('스마트') 웹 사이트를 소개하는 내용만으로 100%로 가득 채워 깔끔하게 출력한다.
3. 푸터(Footer): 다른 내용 없이 주소, 전화번호, 저작권을 간단한 더미 데이터로 생성하여 출력(여백 줄여서)

[UX/UI 디자인 요구사항]

1. 모바일과 데스크톱 화면을 모두 지원하는 반응형 페이지로 작성한다.
2. @DESIGN.md (Claude) 디자인 마크다운 파일을 참고하여 테마를 설정한다.
3. Tailwind CSS를 조합해서 'shadcn/ui' 디자인이랑 똑같이 생긴 '버튼', '카드상자'로 깔끔하게 교체한다.
5. 모든 페이지에서 작업이 오래 걸릴 때는 스피너(Spinner)와 로딩 메시지 출력한다.
6. 아이콘은 'Lucide' 라이브러리를 사용해 홈페이지에 어울리는 그래픽 요소들로 배치한다.
7. 한글 글꼴은 구글 폰트 `[https://fonts.google.com/?lang=ko\\_Kore](https://fonts.google.com/?lang=ko_Kore)`에서 'IBM Plex Sans KR'로 사용한다.

3. 로그인/회원가입 페이지 작성

목표: 사용자 테이블('Users')을 기반으로 로그인/회원가입 페이지를 작성한다.

[회원가입 기능]

1. 아이디와 비밀번호(8자리이상, 영문자, 숫자, 특수문자 반드시 포함), 물(학생/교수), 학생번호/교수번호 입력
2. 비밀번호는 암호화(`bcrypt`)하여 데이터베이스에 저장

[로그인 기능]

1. 아이디와 비밀번호를 입력
2. 로그인 성공 시
 - JWT를 사용하여 로그인 상태 유지, 틀리면 정확한 실패 사유 발생
 - 관리자인 경우 네비게이션바 중앙에 '교수관리', '학생관리', '강좌관리', '수강신청관리' 메뉴 추가
 - 네비게이션바 '로그인' 버튼을 '로그아웃' 버튼으로 변경하고 '로그아웃' 버튼 왼쪽에 학생이름/교수이름/관리자를 출력
3. '로그아웃' 버튼 클릭 시 모달창(헤더: 버튼과 같은 색상)에 '정말로 로그아웃하시겠습니까?' 질의 후 '예'인 경우 로그아웃

[UX/UI 디자인 요구사항]

1. 화면 중앙에 깔끔하고 모던한 카드(Card) 레이아웃 형태로 로그인 폼을 배치
2. 물은 라디오 버튼으로 '교수', '학생'을 선택하여 입력
3. 로그인 폼 하단에 '아직 회원이 아니신가요? 회원가입' 링크를 두어, 클릭 시 회원가입 폼으로 이동
4. 회원가입 폼 하단에 '이미 계정이 있으신가요?' '로그인' 링크를 두어, 클릭 시 로그인 폼으로 이동
5. 로그인/회원가입 시 비밀번호 입력상자에 초기 값으로 'password123!'을 입력

목표: 관리자로 로그인한 경우 히어로 섹션을 숨기고 메인 콘텐츠 내용을 아래 내용으로 변경한다.

[메인콘텐츠 출력내용]

1. 구글 차트(<https://developers.google.com/chart?hl=ko>) CDN 라이브러리를 이용한다.
2. 2행2열의 그리드 레이아웃으로 구성한다.
3. 1행 1열의 그리드 아이템에 학과별 학생 인원수 막대그래프, 1행 2열의 그리드 아이템에 강좌별 평균점수를 막대그래프로 출력한다.
4. 2행 1열의 그리드 아이템에 강좌별 수강 인원수를 원 그래프, 2행 2열의 그리드 아이템에 날짜별 수강신청 인원수를 꺾은선 그래프로 출력한다.

4. 관리자 모드

목표: 관리자 모드에서 교수 테이블(professors)을 기반으로 CRUD 기능을 하는 교수관리 페이지를 작성한다.

[기능 및 요구사항]

1. 교수목록 페이지

- 데스크톱인 경우 표로 출력하고 모바일인 경우 카드로 출력한다.
- 검색 입력상자는 왼쪽에, '교수등록' 버튼은 오른쪽에 배치한다.
- 검색어(교수번호, 교수이름, 교수학과)를 입력한 후 엔터키를 쳤을 때 검색 기능을 수행한다.
- 페이지네이션은 데스크톱인 경우 (itemsPerPage:5, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.
- 마지막 칼럼에 '수정', '삭제'(연관 데이터가 있으면 삭제 불가 + 안내 메시지) 버튼(아이콘 포함)을 배치한다.

2. 교수입력 페이지

- '교수번호'는 교수 테이블의 교수번호 MAX+1 값을 자동 등록한다.
- '교수입력' 폼 오른쪽 상단에 '목록으로' 링크 텍스트를 배치하고 클릭 시 '교수목록' 페이지로 이동한다.
- '저장', '취소' 버튼은 교수정보가 입력 전인 경우 Disable 상태를 유지한다.

3. 교수수정 페이지

- '교수이름', '교수직급', '임용일' 세 칼럼만 수정 가능하도록 한다.
- '교수수정' 폼 오른쪽 상단에 '목록으로' 링크 텍스트를 배치하고 클릭 시 '교수목록' 페이지로 이동한다.
- '저장', '취소' 버튼은 교수정보가 수정 전인 경우 Disable 상태를 유지한다.

[UX/UI 디자인 요구사항]

1. '네비게이션바'의 '교수관리' 메뉴를 클릭하면 '교수목록' 페이지로 이동한다.
2. '교수입력', '교수수정' 페이지
 - 폼은 화면 중앙에 깔끔하고 모던한 카드(Card) 레이아웃 형태로 배치하고 레이블(오른쪽 정렬)과 입력상자를 한 줄로 배치한다.
 - 교수학과, 교수직급은 선택박스로 값을 입력 받는다.
3. '저장', '삭제', '취소' 버튼을 클릭한 경우에 모달창(예: '저장'버튼과 같은 색)을 사용하여 질의 후 '예'인 경우에만 저장/삭제/취소한다.

목표: 관리자 모드에서 학생 테이블(students)을 기반으로 CRUD 기능을 하는 학생관리 페이지(교수관리 참고)를 작성한다.

[기능 및 요구사항]

1. 학생목록 페이지

- 학생목록 표에서 지도교수번호 칼럼에 지도교수이름(지도교수번호)으로 출력한다.
- 학생목록은 학생번호, 학생이름, 학과, 지도교수이름으로 검색하도록 한다.

2. 학생입력 페이지

- '학생번호'는 학생 테이블의 학생번호 MAX+1 값을 자동 등록한다.
- '학생번호' 칼럼이외에 모든 칼럼을 입력 받는다.

3. 학생수정 페이지

- '학생이름', '생년월일', '지도교수번호' 세 칼럼만 수정 가능하도록 한다.

[UX/UI 디자인 요구사항]

1. '학생입력', '학생수정' 페이지의 학생학과는 선택박스로 rltqtd,f 입력 받는다.
2. '지도교수번호'는 교수번호+교수이름을 검색 가능한 선택상자에 출력하고 교수이름 초성, 교수번호로 검색하여 지도교수번호를 입력 받는다.

목표: 관리자 모드에서 강좌 테이블(courses)을 기반으로 CRUD 기능을 하는 강좌관리 페이지(학생관리 참고)를 작성한다.

[기능 및 요구사항]

1. 강좌목록 페이지

- 강좌목록 표에서 담당교수번호 칼럼에 교수이름(담당교수번호)으로 출력한다.
- 강좌목록 강좌번호, 강좌이름, 담당교수이름으로 검색하도록 한다.

2. 강좌입력 페이지

- '강좌번호'는 강좌 테이블의 강좌번호 MAX+1 값을 자동 등록한다.
- '강좌번호', '현재수강인원'이외에 모든 칼럼을 입력 받는다.

3. 강좌수정 페이지

- '강좌이름', '강의시간', '최대수강인원', '담당교수번호' 네 칼럼만 수정 가능하도록 한다.

[UX/UI 디자인 요구사항]

- '담당교수번호'는 교수번호+교수이름을 검색 가능한 선택상자에 출력하고 교수이름 초성, 교수번호로 검색하여 담당교수번호를 입력 받는다.

목표: 관리자 모드에서 수강신청 테이블(enrollments)을 기반으로 목록, 삭제하는 수강신청 목록 페이지를 작성한다.

[기능 및 요구사항]

1. 강좌번호, 강좌이름, 학생번호, 학생이름, 수강신청일, 점수 칼럼들을 출력한다.
2. 강좌번호, 강좌이름, 학생번호, 학생이름으로 검색하도록 한다.
3. 마지막 칼럼에 '수강취소' 버튼(아이콘)을 추가한다.
4. 수강 신청한 강좌들을 전체선택/부분선택(체크박스) 하여 한꺼번에 취소하는 기능을 추가한다.

5. 사용자 모드(교수)

목표: 사용자 모드(교수)로 로그인한 경우 히어로 섹션을 숨기고 메인 콘텐츠를 교수정보 페이지로 변경한다.

[기능 및 요구사항]

1. 교수정보 페이지
 - 교수의 모든 정보를 출력한다.
 - 로그인한 교수가 담당하는 강좌목록을 출력하고 강좌이름을 클릭하면 강좌정보 페이지로 이동한다.
 - 로그인한 교수가 지도하는 학생목록을 출력하고 학생이름을 클릭하면 학생정보(카드사용)를 출력하는 모달창을 띄운다.

2. 강좌정보 페이지

- 강좌의 모든 정보를 출력한다.
- 해당 강좌를 수강 신청한 학생목록을 출력한다.
- 학생목록에는 학생번호, 학생이름, 학생학과, 수강신청일을 출력하고 점수+수정버튼(아이콘포함)은 같은 칼럼에 출력한다.
- 점수들을 전체선택/부분선택(체크박스) 하여 한꺼번에 수정하는 기능을 추가한다.

[UX/UI 디자인 요구사항]

1. 교수 정보와 강좌 정보는 한 줄로 깔끔하게 배치하여 출력한다.
2. 로그인한 교수가 담당하는 강좌목록과 지도하는 학생목록은 탭(Tab)을 이용하여 배치한다.
3. 강좌목록, 학생목록, 수강 신청한 학생목록
 - 데스크톱인 경우 표로 출력하고 모바일인 경우 카드로 출력한다.
 - 페이지네이션은 데스크톱인 경우 (itemsPerPage:3, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.
 - 학생정보 모달창의 '닫기' 버튼을 모달창 헤더 오른쪽 상단에 X 버튼으로 변경한다.
4. 점수 '수정' 버튼을 클릭한 경우에 모달창(헤더: 버튼과 같은 색)을 사용하여 질의 후 '예'인 경우에만 저장한다.

6. 사용자 모드(학생)

수강신청 테이블에서 강좌별 수강신청 학생인원 수를 계산하여 강좌 테이블의 '현재수강인원' 값으로 수정한다.

목표: 사용자 모드(학생)로 로그인한 경우 히어로 섹션을 숨기고 메인 콘텐츠를 학생정보 페이지로 변경한다.

[기능 및 요구사항]

1. 학생의 모든 정보와 교수이름(지도교수번호)을 출력한다.
2. 로그인한 학생이 수강 신청한 강좌목록
 - 강좌번호, 강좌이름, 교수이름(담당교수번호), 수강신청일, 점수, 수강취소 버튼(아이콘 포함)을 출력한다.
 - 수강취소 버튼을 클릭하면 수강신청 테이블에서 해당 강좌를 삭제하고 강좌 테이블의 '현재수강인원' 값을 1감소한다.
3. 수강신청 버튼을 클릭하면 모달창에 강좌목록을 출력한다.
 - 강좌목록에는 강좌번호, 강좌이름, 강의시수, 현재수강인원/최대수강인원, 상태를 출력한다.
 - 이미 신청한 강좌인 경우 상태 칼럼에 '신청완료' 배지를 출력한다.
 - 현재수강인원 >= 최대수강인원인 경우 상태 칼럼에 '마감완료' 배지를 출력한다.
 - 현재수강인원 < 최대수강인원인 경우 상태 칼럼에 '신청가능' 배지를 출력한다.
 - '신청가능' 배지를 클릭 시 수강신청 테이블에서 해당 강좌를 등록하고 강좌 테이블의 '현재수강인원' 값을 1증가한다.

[UX/UI 디자인 요구사항]

1. 수강신청 버튼을 수강 신청한 강좌목록 오른쪽 상단에 배치한다.
2. 페이지네이션은 데스크톱인 경우 (itemsPerPage:3, visiblePages:5), 모바일인 경우 간단한 UI(이전, 다음)를 사용한다.
3. 수강취소 버튼을 클릭한 경우 모달창(헤더: 버튼과 같은 색)을 사용하여 질의 후 '예'인 경우에만 수강취소 작업을 한다.
4. 강좌목록 모달창의 '닫기' 버튼을 모달창 헤더 오른쪽 상단에 X 버튼으로 변경한다.
5. 알림을 토스트(내용은 가운데 정렬)로 바꿔서 화면 아래에 출력한다.

• 프로젝트 추가 기능

1. 자기소개 홈페이지

[추가 요구사항]

1. Light, Dark 모드를 클릭한 경우 '정말로 변경하실래요?' 모달창을 띄워 '예'인 경우에만 변경한다.
2. 모달창 헤더의 배경색과 글자색은 기본 버튼 테마를 적용한다.
3. 모든 한글 글꼴은 https://fonts.google.com/specimen/Sunflower?lang=ko_Kore&preview.lang=ko_Kore Sunflower로 변경한다.
4. 이름 왼쪽에 30대 남성 프로필 사진을 생성해서 예쁘게 넣는다.
5. 프로필 이미지를 클릭하면 모달창에 큰 이미지를 출력한다.

2. 과일가게 홈페이지

목표 아래 요구사항을 구현하는 장바구니 페이지를 생성 후 장바구니 메뉴와 연결한다.

[요구사항]

1. 장바구니 목록을 출력한다.
2. 장바구니 아이템의 수량을 수정하는 기능을 추가한다.
3. 장바구니 아이템을 삭제하는 기능을 추가한다.
4. 삭제하는 경우 '정말로 삭제하시겠습니까?' 모달창을 띄워 예인 경우에만 삭제한다.

[디자인 요구사항]

1. 모달창의 헤더의 배경색과 글자색은 기본 버튼 색상과 동일하게 설정한다.
2. 장바구니 페이지에 메인 페이지 히어로 섹션 영역을 출력한다.

3. 인테리어 홈페이지

[추가 요구사항]

1. 시공사례 이미지를 검색하여 다운로드하지 않고 링크로 출력한다.
2. 우리 홈페이지와 어울리는 이미지를 검색하여 다운로드하지 않고 히어로섹션 배경이미지에 링크로 출력한다.
3. 최적화된 히어로 이미지(.webp) 5개를 검색하여 다운로드한 후 슬라이드로 미끄러지듯이 출력한다.
4. 히어로 이미지 좌, 우 이동 버튼을 사이즈를 키워 예쁜 커스텀(Custom) 버튼으로 변경한다.

[디자인 요구사항]

1. @DESIGN.md 변경된 파일을 참고하여 디자인을 새로 적용한다.
2. 히어로 섹션 글자 색상을 이미지와 잘 어울리도록 변경한다.

4. 게임 홈페이지

[알카노이드 게임 추가 요구사항]

- 벽돌 이미지를 검색하고 붙여 놓은 후 이 이미지를 참고해서 벽돌이미지를 변경한다.

[테트리스 게임 요구사항]

1. 메인 콘텐츠를 3열로 변경하고 '테트리스 게임' 카드상자를 추가한다.
2. 테트리스 게임 규칙을 검색하여 'tetris_game.md' 마크다운 파일에 저장한다.
3. @tetris_game.md 파일을 참고하여 게임 프로그램을 작성하여 '테트리스 게임' 카드상자와 연결한다.

5. 데이터베이스

[추가 요구사항]

1. 메모앱 메모 목록의 내용들은 2줄만 출력하고 2줄이 넘는 경우에는 생략기호를 출력한다.
2. 메모 목록의 각 메모 정보에 수정, 삭제 아이콘을 작성일과 같은 행 오른쪽에 추가한다.
3. 수정 아이콘을 클릭하면 라지 사이즈 모달창을 띄워 내용과 태그를 수정하는 기능을 추가한다.
4. 메모 수정 모달창에서 내용이나 태그가 수정되기 전에는 저장 버튼을 Disabled 상태로 유지한다.
5. 모든 페이지 넓이는 메인 콘텐츠 넓이와 같게 설정한다.

6. 산악회 커뮤니티

1) 게시물에 대한 댓글관리 프로그램을 작성하고 게시물 상세 페이지에 출력한다.

목표: 게시물에 댓글 관리 페이지를 작성하고 게시물 상세 페이지에 출력한다.

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 'PostComments' 테이블 스키마를 정의한다.
2. 테이블 필드: id, postId(게시글 id), writer(작성자:nickname), content(내용) createdAt(작성일), updatedAt(수정일)

[댓글 목록 기능 및 UX/UI 요구사항]

1. 최근에 작성된 댓글 순으로 출력하고 댓글이 많은 경우 무한 스크롤링 기능을 추가한다.
2. 상단 왼쪽에 작성자, 작성일, 오른쪽에는 로그인한 작성자가 쓴 댓글인 경우 '수정', '삭제' 링크 텍스트를 출력한다.
3. 댓글 내용은 3줄만 출력하고 3줄 이상인 경우 '자세히', '간략히' 링크 텍스트 출력하고 토글(Toggle) 기능을 추가한다.

[댓글 등록 기능 및 UX/UI 요구사항]

1. 로그인한 사용자용 댓글 입력창(textarea-autosize)을 출력한다.
2. '취소', '등록' 버튼은 입력내용이 없는 경우에는 Disabled 상태를 유지한다.

[댓글 삭제 기능 및 UX/UI 요구사항]

- 댓글을 삭제할 때는 '정말로 삭제하시겠습니까?' 모달창을 띄운 후 '예'인 경우에만 삭제한다.

[댓글 수정 기능 및 UX/UI 요구사항]

1. 내용을 수정할 수 있는 입력창(textarea-autosize)으로 변경하고 커서를 맨 아래 줄에 위치한다.
2. '저장' 버튼은 수정되지 않은 경우에는 Disabled 상태를 유지한다.
3. 댓글을 수정하는 경우 '정말로 수정하시겠습니까?' 모달창을 띄운 후 '예'인 경우에만 수정한다.

2) 모든 프로젝트에서 공통으로 사용할 댓글 관리 스킬을 생성한 한다.

/create-skill

1. Skill name: **comments-skill-builder**
2. Description: 다양한 프레임워크에 맞춰 댓글 CRUD 작업을 수행하기 위한 모든 프로젝트에서 사용할 개인 스킬이다.
3. Trigger Keywords: 댓글, 댓글 관리, Comments
4. Skill Prompt (AI 행동 지침)

[데이터베이스 요구사항]

1. Prisma와 SQLite을 연동하여 현재 데이터베이스에 부모 테이블+'Comments' 테이블 스키마를 정의한다.
2. 테이블 필드: id, 부모 테이블+id writer(작성자:nickname), content(내용) createdAt(작성일), updatedAt(수정일)

[댓글 목록 기능 및 UX/UI 요구사항]

1. 최근에 작성된 댓글 순으로 출력하고 댓글이 많은 경우 무한 스크롤링 기능을 추가한다.
2. 상단 왼쪽에 작성자, 작성일, 오른쪽에는 로그인한 작성자가 쓴 댓글인 경우 '수정', '삭제' 링크 텍스트를 출력한다.
3. 댓글 내용은 3줄만 출력하고 3줄 이상인 경우 '자세히', '간략히' 링크 텍스트 출력하고 토글(Toggle) 기능을 추가한다.

[댓글 등록 기능 및 UX/UI 요구사항]

1. 로그인한 사용자용 댓글 입력창(textarea-autosize)을 출력한다.
2. '취소', '등록' 버튼은 입력내용이 없는 경우에는 Disabled 상태를 유지한다.

[댓글 삭제 기능 및 UX/UI 요구사항]

- 댓글을 삭제할 때는 '정말로 삭제하시겠습니까?' 모달창을 띄운 후 '예'인 경우에만 삭제한다.

[댓글 수정 기능 및 UX/UI 요구사항]

1. 내용을 수정할 수 있는 입력창(textarea-autosize)으로 변경하고 커서를 맨 아래 줄에 위치한다.
2. '저장' 버튼은 수정되지 않은 경우에는 Disabled 상태를 유지한다.
3. 댓글을 수정하는 경우 '정말로 수정하시겠습니까?' 모달창을 띄운 후 '예'인 경우에만 수정한다.

3) 댓글 관리 스킬을 사용하여 산행 일정과 활동 후기에 적용한다.

산행 일정에 대한 댓글 관리 페이지를 작성하고 산행 일정 상세 페이지에 출력한다.

활동 후기에 대한 댓글 관리 페이지를 작성하고 활동 후기 상세 페이지에 출력한다.